

Notas del curso
Python para el Cómputo Científico

Curso de Actualización Académica de la DGAPA-UNAM
Facultad de Ciencias

Dr. David P. Sanders

5 de agosto de 2013

© David P. Sanders, 2013

II

©

Índice general

0. ¿Qué es Python?	v
0.1. Meta del curso	v
 I El lenguaje básico	 1
 1. Comenzando con Python	 3
1.1. El entorno <code>ipython</code>	3
1.2. Aritmética	4
1.3. Variables	4
1.4. Entrada por parte del usuario	5
1.5. Listas	6
1.6. n -adas / tuples	6
1.7. Diccionarios	7
1.8. Programitas de Python: scripts	7
 2. Estructuras de control	 9
2.1. Bucle <code>for</code>	9
2.2. Condicionales	10
2.3. Bucle <code>while</code>	10
2.4. “Comprensiones” de listas	11
 3. Funciones y bibliotecas	 13
3.1. Funciones	13
3.2. Bibliotecas	14
3.3. La biblioteca estándar	15
3.4. Bibliotecas de matemáticas	15
3.5. Argumentos de funciones	17
3.6. ¿Nombres o variables?	17
3.7. Conversión de números	18

3.8. Aritmética con precisión arbitraria	18
3.8.1. ¿Cuánto tiempo tarda una función?	19
3.9. Sustitución de variables en cadenas	19
3.10. Nueva sintaxis para sustitución de variables en cadenas	20
4. Archivos y procesos	21
4.1. Redirección de la entrada y salida estándar	21
4.2. Argumentos de la línea de comandos	21
4.3. Leer archivos	22
4.4. Escribir archivos	23
4.5. Interacción con otros programas (procesos)	24
II Paquetes para el cómputo científico	27
5. La biblioteca numpy para vectores y matrices	29
5.1. Necesidad de una biblioteca especializada para cómputo científico	29
5.2. Operando con vectores	30
5.3. Extrayendo partes de un vector	30
5.4. Creando vectores de forma rápida	31
5.5. Funciones de vectores	31
5.6. Matrices	32
5.7. Broadcasting	33
5.8. Comparación de arrays	34
5.9. Funciones para resumir información sobre vectores y matrices	34
5.10. Multiplicación de matrices y vectores	34
5.11. Leer un array desde un archivo	35
5.12. Escribir un array a un archivo	35
6. Submódulos de numpy para cómputo científico: álgebra lineal, transformadas de Fourier y números aleatorios	37
6.1. Álgebra lineal	37
6.2. Transformadas de Fourier	38
6.3. Números aleatorios	38
7. Gráficas 2D con matplotlib	41
7.1. Entorno pylab	41
7.2. Gráficas básicas	41
7.3. Cambiando el formato	42

7.4. Etiquetas	42
7.5. Figuras desde scripts	43
7.6. Escalas logarítmicas	44
7.7. Múltiples dibujos (subplot)	44
7.8. Otros tipos de gráficas	45
8. Matplotlib avanzado	47
8.1. Matplotlib para dibujar figuras	47
8.2. Campos escalares y vectoriales: Mallas con meshgrid	48
8.3. Campos vectoriales	49
8.4. Gráficas en 3D	50
8.5. Animaciones	50
8.6. Interacción con el mouse y el teclado	51
9. Animaciones sencillas con Visual Python	53
9.1. El paquete Visual Python	53
9.2. Animaciones	53
9.3. Agregando propiedades a objetos	54
9.4. La escena	54
10. Cálculos de cómputo científico con scipy	55
10.1. Álgebra lineal	55
10.2. Cálculos numéricos con <code>scipy</code>	55
10.3. <code>pygsl</code>	57
11. Visualizaciones en 3D con <code>mayavi2</code>	59
11.1. Usando <code>mayavi2</code> de manera interactiva	59
11.2. Comandos básicos	60
11.3. Animaciones	60
11.4. Ejemplo: visualizar un campo escalar en 3D	61
III Programación orientada a objetos	63
12. Programación orientada a objetos: clases	65
12.1. El problema: variables que están relacionadas conceptualmente, pero no en el código	65
12.2. La solución: encapsulación de datos en objetos (“clases”)	66
12.3. Métodos: acciones que llevan a cabo los objetos	67
12.4. Función inicializadora (“constructor”)	67

12.5. Métodos internos (escondidos) de las clases	68
12.6. Partículas en n dimensiones	69
12.7. Herencia	69
IV Otros paquetes y herramientas	71
13. Cálculo simbólico con Python	73
13.1. El paquete <code>sympy</code>	73
13.2. Entorno completo: <code>SAGE</code>	74
14. Depurador y perfilador	77
14.1. Depurador	77
14.2. Perfilador	77
15. Otros paquetes	79
15.1. Notebook de IPython	79
15.2. <code>pandas</code>	79
15.3. Operaciones con objetos en <code>pandas</code>	80

Capítulo 0

¿Qué es Python?

Python es un lenguaje de programación que surgió en 1990. Desde entonces se ha desarrollado enormemente, para volverse un lenguaje de programación moderno, y uno de los más utilizados en el mundo.

Python es un lenguaje *interpretado* –no es necesario compilar constantemente cada programa, si no se puede utilizar de manera *interactiva*. Por lo tanto, es una herramienta idónea para llevar a cabo tareas de cómputo científico, tales como análisis de datos, graficación de resultados, y exploración de problemas. En este sentido, se puede comparar con programas comerciales tales como Matlab, pero con la diferencia que es software libre (es decir, se puede acceder al código fuente, modificarlo y distribuirlo) y disponible gratis. También hay paquetes que permiten llevar a cabo cálculos simbólicos a través de Python, de los cuales destaca *sage* (www.sagemath.org), por lo cual también se puede considerar como competidor de Mathematica y Maple.

Python viene con una filosofía de “baterías incluidas”. Esto quiere decir que hay muchas bibliotecas disponibles que están diseñadas para facilitarnos la vida al hacer diferentes tareas, desde el cómputo científico hasta la manipulación de páginas web. Por lo tanto, Python es un lenguaje sumamente versátil.

Cabe enfatizar que se suele encontrar Python fácil de aprenderse y de utilizarse. Por lo tanto, considero que Python es también el lenguaje idóneo para la enseñanza del cómputo científico en la Facultad de Ciencias. Éste es un campo que se ha vuelto de suma importancia, y que todavía no se ve reflejado adecuadamente en la carrera.

0.1. Meta del curso

La meta principal de este curso es la de explorar las técnicas básicas de Python en el contexto del cómputo científico, y en particular de la física computacional. El fin es el mejoramiento de la docencia del cómputo científico en la UNAM.

Parte I

El lenguaje básico

Capítulo 1

Comenzando con Python

En este capítulo, empezaremos a explorar el lenguaje Python. Se darán las bases tanto de la sintaxis básica, como de las herramientas principales para escribir programas en el lenguaje. En todo el curso se hará énfasis en las aplicaciones al cómputo científico y a la enseñanza del mismo.

La versión de Python que ocuparemos es la 2.7. La reciente versión 3 tiene algunas diferencias de sintaxis, las cuales rompen la compatibilidad con los paquetes básicos para el cómputo científico por el momento.

`python` es un programa que provee el intérprete del lenguaje Python, es decir, que lee el código fuente y lleva a cabo las acciones correspondientes. Se puede utilizar tanto de manera interactiva, como a través de un “script” (un programa que consiste en una secuencia de comandos de Python que se ejecutan.)

1.1. El entorno `ipython`

Es necesario contar con un entorno de trabajo adecuado. Hay muchas opciones, pero una muy buena, y la principal que ocuparemos para el uso interactivo, es `ipython`. También hay distintos entornos estilo IDE (Entorno de Desarrollo Interactivo), de los cuales destaca `spyder` (para Linux y Mac); incluye la posibilidad de utilizar `ipython` como su “shell” interactivo.

`ipython` es un programa que nos permite interactuar de manera cercana con el intérprete de Python, el cual interpreta los comandos que mandamos; está diseñado para maximar la productividad al utilizar Python. En particular, tiene ciertos comandos para facilitarnos la vida que no existen en otros intérpretes de Python. Estos comandos “mágicos” empiezan con `%` (aunque a menudo no es necesario colocar el `%`). Uno muy útil es

```
In [1]: %logstart clase1_log.py
```

Este comando graba una bitácora (“log”) de todo lo que se haya tecleado y lo que se teclee en la sesión actual en el archivo dado, aquí `clase1_log.py`.

Otra función de suma utilidad en `ipython` es la de completar palabras parciales con la tecla <TAB> (tabulador). Además, los comandos `%who` y `%whos` proveen información de las funciones nuevas agregadas en la sesión. Nótese también que las flechas funcionan para regresar a comandos tecleados anteriormente. (También hay una función mágica `%history`.)

Una lista de todas las funciones mágicas se obtiene con `%lsmagic`, y más información con `%magic`.

La documentación para cualquier función o comando está disponible en `ipython` al poner `?`, por ejemplo

```
who?
```

Para funciones, `??` muestra el código fuente de la función, cuando esto está disponible (es decir, cuando la función está escrita en Python, y no en C).

La historia de los comandos que hayas tecleado se puede obtener con `%history`.

Una versión más cómoda del interfaz de `ipython` se puede obtener al correrlo con una opción: `ipython qtconsole` y `ipython notebook` (una “libreta” en formato HTML). Luego veremos un entorno para graficar, para el cual es conveniente correrlo al menos con la opción `ipython --pylab`.

El entorno integrado de desarrollo `spyder` permite utilizar `ipython` para correr programas y manipular los resultados.

1.2. Aritmética

La tarea más básica del cómputo científico es el llevar a cabo cálculos numéricos. Python se puede utilizar como una calculadora:

```
3
3 + 2
3 * (-7.1 + 10**2)
```

Aquí, `**` indica una potencia, como en Fortran, y las paréntesis alteran la prioridad en las operaciones.

¿Cuál es la diferencia entre $3 \times 4 + 2$ y $3 + 4 \times 2$? ¿Qué operación tiene prioridad?

Ejercicio: Es necesario¹ tener cuidado al hacer manipulaciones con enteros:

```
1 / 2
3 / 2
5 / 2
```

ya que el resultado se redondea al entero más cercano. Para evitar eso, basta un punto decimal:

```
1 / 2.
```

El punto en la última sentencia convierte el número 2 en un número real, de tipo “flotante”. Los cálculos con flotantes funcionan como esperamos en general, pero no siempre:

```
3.2 * 1.8
```

El problema aquí radica en la representación interna de los números reales en la computadora con precisión finita. No se puede evitar al trabajar con números flotantes, en ningún entorno.

Los números sin punto decimal se consideran enteros, y los con punto decimal flotantes (de doble precisión). Los enteros pueden ser *arbitrariamente grandes*:

```
2 ** 2 ** 2 ** 2 ** 2 ** 2
```

Aquí, como en Fortran, `**` denota exponenciación.

Python cuenta con números complejos, utilizando la notación `1j` para la raíz de -1 :

```
1 + 3j
1j * 1j
j      # da un error
```

‘`#`’ indica un comentario que se extiende hasta el final de la línea.

1.3. Variables

Para llevar a cabo cálculos, necesitamos *variables*, que se pueden pensar como lugares en la memoria en donde guardar información que se puede recuperar a través del nombre de la variable.

Las variables se declaran como debe ser:

¹Es necesario en Python 2.7; la situación cambia en Python 3. Para tener el nuevo comportamiento se puede hacer `from __future__ import division`. Nótese que hay dos guiones bajos de cada lado.

```
a = 3
b = 17.5
c = 1 + 3j

print (a + b / c)
```

El comando `print` imprime su argumento de una manera bonita. En Python 2.7, se usa así:

```
print "El resultado es ", a+ b/ c
```

En Python 3, se requieren paréntesis²:

```
print ("El resultado es ", a+ b/ c)
```

Para tener este comportamiento en versiones anteriores, se puede usar `from __future__ import print_function`.

Python reconoce de manera *automática* el tipo de la variable según su forma; al reasignar una variable, se pierde la información de su valor interior, incluyendo el tipo. El tipo actual de una variable se puede averiguar con la función `type`:

```
a = 3
type(a)
a = -5.5
type(a)
a = "Hola"
type(a)
```

1.4. Entrada por parte del usuario

Si es realmente necesario, se puede pedir información del usuario con

```
a = raw_input('Dame el valor de a: ')
a      # convertir la cadena a a float
print "El cuadrado de a es ", a*a
```

Nótese que la respuesta es una *cadena*; las cadenas en Python son cadenas de caracteres entre pares de apóstrofes o comillas. Si queremos una versión como número, entonces tenemos que modificar el tipo:

```
a = raw_input('Dame el valor de a: ')
a = float(a)      # convertir la cadena a a float
print "El cuadrado de a es ", a*a
```

o directamente sin guardar el valor intermedio.

```
a = float( raw_input('Dame el valor de a: ') )
a      # convertir la cadena a a float
print "El cuadrado de a es ", a*a
```

Sin embargo, suele ser una mala idea pedir información de esta forma, ya que interrumpe el flujo de trabajo. Hay otras maneras de llegar al mismo efecto de proveerle información a un programa.

Ejercicio: Pide del usuario un radio, y calcula el volumen de la esfera con dicho radio, imprimiendo el resultado de manera bonita.

²`print` se vuelve entonces una *función*; ver el capítulo correspondiente.

1.5. Listas

Para hacer cómputo científico, necesitamos tener la posibilidad de guardar muchos datos en una estructura razonable, en lugar de uno por uno en variables con distintos nombres.

Una de las dos estructuras de datos principales para en Python es la *lista*. Consiste literalmente en una lista ordenada de cosas, y reemplaza –hasta cierto punto– a los arreglos en otros lenguajes. La diferencia es que las listas en Python son automáticamente de longitud *variable*, y pueden contener objetos de *cualquier* tipo; o sea, que son mucho más flexibles y útiles que en la mayoría de los demás lenguajes.

Una lista se define con corchetes (`[ ]`):

```
l = [3, 4, 6]
```

Puede contener cualquier cosa, ¡incluyendo a otras listas!:

```
l = [3.5, -1, "hola", [1.0, [3, 17]]]
```

Por lo tanto, es una estructura de datos muy versátil.

Los elementos de la lista se pueden extraer y cambiar usando la notación `l[i]`, donde `i` indica el número del elemento, empezando desde 0:

```
l = [1, 2, 3]
print l[0], l[1]
l[0] = 5
l
```

Se pueden manipular rebanadas (“slices”) de la lista con la notación `l[1:3]`:

```
l = [1, 2, 3, 6, 7]
l[1:3]
l[:3]
l[3:]
```

Ejercicio: ¿Qué hace `l[-1]`?

La longitud de una lista se puede encontrar con

```
len(l)
```

Se pueden agregar elementos a la lista con

```
l = [] # lista vacia
l.append(17)
l.append(3)
print l, len(l)
```

Las otras operaciones que se pueden llevar a cabo con la lista se pueden averiguar en `ipython` con `l.<TAB>`.

Aquí, vemos los primeros indicios de que en Python *todo es un objeto*. Un objeto podemos considerarlo simplemente como una cosa que tiene propiedades; estas propiedades, que son propias del objeto (o, más bien, del tipo correspondiente al objeto) pueden ser variables internas y funciones propias (llamadas *métodos*). Por ejemplo, `l.append` es el nombre de una función propia a la clase de objetos llamada ‘lista’; es una función que agrega un elemento al final de la lista. [Véase el capítulo sobre funciones.]

1.6. *n*-adas / tuples

Otra estructura parecida a una lista es una *n*-ada (“tuple”). Se escribe con (o incluso, a veces, sin) paréntesis, y no se puede modificar:

```
t = (3, 5)
t[0]
t[0] = 1          # error!
```

Las *n*-adas se utilizan para agrupar información. Se pueden asignar “todo en uno” como sigue:

```
a, b = 3, 5
print a; print b
```

1.7. Diccionarios

Un diccionario es el otro tipo fundamental de estructura de datos en Python. Los diccionarios son *arreglos asociativos*; es decir, asocian un valor a otro. Un ejemplo clásico le asigna a cada elemento su peso atómico:

```
pesos = {}
pesos["H"] = 1
pesos["Li"] = 3
pesos["Cl"] = 35.5

pesos["He"]      # da error
pesos            # imprime el diccionario
```

Python utiliza los diccionarios internamente, por lo cual están altamente optimizados para inserción y para búsquedas. Están implementados en términos de un “hash map”.

1.8. Programitas de Python: scripts

Una vez que las secuencias de comandos se complican, es útil poder guardarlos en un programa, o *script*. Eso consiste en un archivo de texto, cuyo nombre termina en `.py`, donde se guarda la secuencia de comandos.

Por ejemplo, guardemos el código que calcula el volumen de una esfera en el archivo `volumen_esfera.py`. Estando ya en `ipython`, podemos correr el código con

```
run volumen_esfera
```

(Acordémonos que `ipython` te ayuda incluso a completar nombres de archivos, al teclear la tecla <TAB>.) Al terminar de correr el programa, el control regresa a `ipython`, pero todavía tenemos acceso a las variables (y luego funciones) definidas en el script.

Alternativamente, podemos tratar el script como un programa en sí al ejecutar desde el shell (afuera de `ipython`)

```
python volumen_esfera.py
```

Al terminar, el control regresa al shell, y perdemos la información adentro del script.

De hecho, un script se puede considerar un módulo en sí, que también se puede importar:

```
from cuad import *
```


Capítulo 2

Estructuras de control

Por estructuras de control, se entienden los comandos tales como condicionales, bucles, y funciones, que cambian el orden de ejecución de un programa.

Las estructuras de control tienen un formato en común. A diferencia de C++ y Fortran, no existen instrucciones que indiquen donde empieza y termina un *bloque* de un programa. (Un bloque es un grupo de varias líneas de código que actúan en algún sentido como un solo comando, por ejemplo el cuerpo de un bucle o de una función.) En Python, un bloque empieza por `:`, y su extensión se indica por el uso de una cantidad definida de *espacio en blanco* (*indentación* o *sangría*) al principio de cada línea. Un buen editor de texto permite hacer eso de manera automática. Nótese que cada línea que forma parte del bloque debe tener, a fuerzas, la misma cantidad de espacio en blanco.

2.1. Bucle `for`

La parte central de muchos cálculos científicos consiste en llevar a cabo *bucles* o *ciclos*. Aunque en Python (como en Matlab), esto se se desenfatisa, es crucial saber cómo llevarlos a cabo.

Hay dos tipos de bucle en Python: `for` y `while`, que se ocupan para distintos propósitos.

Un bucle de tipo `for` suele ocuparse para llevar a cabo un número de repeticiones o iteraciones que se conoce de antemano. En el bucle de tipo `for`, encontramos una diferencia importante con respecto a lenguajes más tradicionales: la iteración se lleva a cabo sobre el contenido de una *lista* y ejecutar un bloque de código *para* cada elemento de la lista:

```
l = [1, 2.5, -3.71, "hola", [2, 3]]
for i in l:
    print 2*i
```

Ejercicio: ¿Qué hace `2*x` cuando `x` es una cadena o una lista? ¿Tiene sentido que se hace esto?

Si queremos iterar sobre muchos elementos, es más útil construir la lista de manera automática. Por ejemplo, para hacer una iteración para todos los números hasta 100, podemos utilizar una función útil que provee Python, `range`, que construye una lista en un rango dado:

```
range(10)
```

¿Qué es lo que hace la función `range`? Para averiguarlo, lo investigamos de manera interactiva con `ipython`:

```
range(10)
range(3, 10, 2)
```

Nótese que `range` no acepta argumentos de punto flotante.

Ejercicio: Toma una lista, y crea una nueva que contiene la duplicación de cada elemento de la lista anterior. Ahora podemos iterar sobre esta lista con la sintaxis siguiente:

```
l = range(10)
for i in l:
    print 2*i
```

No es necesario definir una variable –se puede hacer todo al mismo tiempo. Por ejemplo, para imprimir una table (no muy bonita) de los cuadrados de los primeros diez números y sus sumas, podríamos hacer lo siguiente:

```
suma = 0
for i in xrange(1, 11):
    cuadrado = i*i
    suma += cuadrado
    print i, cuadrado, suma
```

Nótese que en este caso no es necesario construir la lista completa al principio. Para esto existe otra función, **xrange**, que construye la lista de manera “floja” [“lazy”] –construye un número a la vez del rango, lo cual es más eficiente si la lista sería muy grande.

Nótese también que todo el código que va indentado (sangrado) con la misma cantidad de espacio conforme el cuerpo del bucle.

2.2. Condicionales

Para checar una condición y ejecutar código dependiente del resultado, se ocupa **if**:

```
a = input('Dame el valor de a')
if a > 0:
    print "a es positiva"
    a = a + 1
elif a == 0:                # NB: dos veces = para checar igualdad
    print "a es 0"
else:
    print "a es negativa"
```

En el primer caso, hay dos comandos en el bloque que se ejecutará en el caso de que $a > 0$. Nótese que **elif** es una abreviación de **else:if**

Las condiciones que se pueden ocupar incluyen: $<$, $<=$ y $!=$ (no es igual). Para combinar condiciones se ocupan las palabras **and** y **or**:

```
a = 3; b=7
if a>1 and b>2:
    print "Grandes"
if a>0 or b>0:
    print "Al menos uno positivo"
```

2.3. Bucle while

Otro tipo de bucle es un **while**, que ejecuta un bloque de código *mientras* una cierta condición se satisfaga, y suele ocuparse para llevar a cabo una iteración en la cual *no* se conoce de antemano el número de iteraciones necesario. Las condiciones tienen la misma sintaxis que para un **if**.

El ejemplo más sencillo de un **while**, donde sí se conoce la condición terminal, para contar hasta 9, se puede hacer como sigue:

```
i = 0
while i < 10:
    i += 1      # equivalente a i = i + 1
    print i
```

Nótese que si la condición deseada es del tipo “hasta...”, entonces se tiene que utilizar un **while** con la condición opuesta.

Ejercicio: Los números Fibonacci se definen como $a_0 = 1$, $a_1 = 1$ y por la recurrencia $a_{n+2} = a_{n+1} + a_n$. Encuentra los números de Fibonacci menores que 1000.

Ejercicio: Implementa el llamado método babilónico para calcular la raíz cuadrada de un número dado y . Este método consiste en la iteración $x_{n+1} = \frac{1}{2} [x_n + (y/x_n)]$. ¿Qué tan rápido converge?

Ejercicio: Pon el código para calcular la raíz cuadrada en un script. ¿Cómo podrías calcular la raíz de los números de 0 hasta 10 en pasos de 0.2? Hazlo e imprime los resultados. [Nota que el código que se pone feo y difícil de entender. Por eso usaremos funciones; ver el capítulo siguiente.]

2.4. “Comprensiones” de listas

Python tiene una forma muy intuitiva de crear listas definidas por una sentencia que involucra sus elementos, por ejemplo “los cuadrados de los números de 1 a 10. En matemáticas, podríamos escribir algo así:

$$\{x^2 : x \in \{1, \dots, 10\}\}. \quad (2.1)$$

En Python es muy parecido:

```
[x*x for x in xrange(0, 11)]
```

También se pueden excluir ciertos valores: por ejemplo, los cuadrados de los números que están divisibles entre 2 o 3 se obtienen con:

```
[x*x for x in xrange(0, 11) if x%2==0 or x%3==0]
```

Aquí, hemos usado el operador `%` que significa aritmética modulo el segundo argumento.

Capítulo 3

Funciones y bibliotecas

3.1. Funciones

Las funciones se pueden considerar como *subprogramas* que ejecutan una tarea dada. Se utilizan para separar partes del código que son independientes. Promueven el re-uso del código. En particular, se deben utilizar cada vez que encuentres código repetido en un programa.

Las funciones en los lenguajes de programación son parecidas a las funciones matemáticas, excepto que: pueden o no aceptar argumentos (0, 1 o múltiples argumentos; e incluso un número indefinido); y pueden o no regresar resultados (0, 1 o más de 1).

La sintaxis para declarar una función es la siguiente:

```
def cuad(x):  
    print "Argumento x = ", x  
    return x*x
```

Hasta este punto, sólo se ha *declarado* o *definido* la función –no se ha ejecutado todavía. Para *llamar* a la función, se usa su nombre, junto con sus *argumentos* (la información que se envía a la función) entre paréntesis:

```
f(3)           # llama a la funcion y le envia el valor 3.  
x = f(3.5)     # llama a la funcion y guarda el resultado en x
```

La palabra **def** introduce la definición, y como siempre el cuerpo de la función es un bloque sangrado. La palabra **return** regresa información al programa principal.

Si no ponemos las paréntesis, entonces Python regresa el objeto que es la función; no llama a la función. Por lo tanto, para llamar a una función sin argumentos, también tenemos que poner paréntesis:

```
def saludo():  
    print "Hola"
```

```
saludo  
saludo()
```

Las funciones se pueden utilizar con argumentos de *cualquier* tipo –el tipo de los argumentos *nunca* se especifica. Si las operaciones llevadas a cabo no se permiten para el tipo que se provee, entonces Python regresa un error:

```
f("hola")
```

Pero si sí están permitidas, entonces funciona perfectamente bien:

```
def duplicar(x):
    return 2*x
```

```
duplicar("Hola")
```

En Python se puede regresar más de un argumento al usar una tupla:

```
def potencias(x):
    return x**2, x**3, x**4
```

```
p = potencias(2)
p
```

Se puede proporcionar una forma sencilla de documentación de una función al proporcionar un “docstring”:

```
def cuad(x):
    """Lleva su argumento al cuadrado.
    NB: Funciona solo si el tipo de x permite multiplicacion.
    """
    return x*x
```

Esto utiliza una cadena de varias líneas, definida por tres comillas """.

Ahora si interrogamos al objeto cuad con `ipython`:

```
cuad?
```

nos proporciona esta información. Si la función está definida en un archivo, entonces `cuad??` muestra el código de la definición de la función.

Sin embargo, para una función así de sencilla, el código de la función se “auto-explica”, por lo cual no deberíamos tener que vernos obligados a proporcionar un docstring. [Ver el libro “Clean Code”, de Robert C. Martin, altamente recomendado.]

3.2. Bibliotecas

Una *biblioteca* (también conocida como “librería”, “paquete”, “módulo”, etc.) es simplemente un archivo de Python que contiene definiciones de funciones. Estas definiciones se pueden *importar* a nuestro código, con la palabra `import`, para que podamos aprovechar estas funciones, sin tener que reescribirlas cada vez. Una gran parte del poder de Python se debe a que cuenta con una gama amplísima de bibliotecas ya disponibles.

Sin embargo, también podemos fácilmente producir nuestras propias bibliotecas, al unir funciones relacionadas en un archivo de Python, por ejemplo

```
def cuad(x):
    return x*x

def cubo(x):
    return x*x*x

def raiz(x):
    return x**0.5
```

Si ponemos este código en un archivo `potencias.py`, entonces podremos volver a utilizar estas funciones en cualquier programa, sin tener que volver a definir las, siempre y cuando importemos la biblioteca correspondiente. Así que en el programa de Python que estamos escribiendo, pondremos

```
import potencias
```

```
potencias.cuad(3)
potencias.raiz(17)
```

Nótese que en la versión más básica de la importación, es necesario poner cada vez el nombre completa de la biblioteca.

3.3. La biblioteca estándar

Python tiene una biblioteca estándar amplia, lo cual siempre está disponible en cualquier instalación de Python. La documentación para esta biblioteca está disponible en <http://docs.python.org/library/>

Por ejemplo, la sección 9 incluye información sobre el módulo `fractions`:

```
from fractions import Fraction # Fraction es funcion definida en el modulo fractions
```

```
a = Fraction(3, 5)
b = Fraction(1)
c = a + b
d = Fraction(4, 6)
d
e = a + d
```

Cada ‘cosa’ en Python es un *objeto*, que tiene propiedades y operaciones disponibles. `Fraction` es un tipo de objeto (*clase*); sus operaciones están disponibles en `ipython` a través de `Fraction.<TAB>`. Las que empiezan con `__` son internos y deberían de ignorarse por el momento; las demás son accesibles al usuario.

Las paréntesis corresponden a objetos que son *funciones* –pueden tomar argumentos y regresar información. En este caso, la función `Fraction` es un “constructor” que construye un objeto de un tipo particular, a partir de los argumentos que se le manden.

3.4. Bibliotecas de matemáticas

Si intentamos calcular funciones más avanzadas, nos topamos con un problema:

```
sin(0.5)
```

nos produce un error que dice que la función `sin` no está definida.

Para utilizar la gran mayoría de las funciones disponibles en el “ecosistema” de Python, es necesario importar una biblioteca, que es una colección de funciones pre-definidas. Por ejemplo, `sin` está definida en la biblioteca que se llama `math`, parte de la biblioteca estándar de Python (lo cual, por lo tanto, debería estar disponible en cualquier máquina en la que está instalado Python) que se importa como sigue:

```
import math

math.sin(0.5)
```

Nótese que el nombre de la función a fuerzas tiene que llevar el prefijo `math.`, ya que proviene de aquella biblioteca. No es necesario que nosotros sepamos en donde exactamente está almacenada la biblioteca en el disco; Python se preocupa de esto solo, en cuanto a la biblioteca estándar se refiere.

En `ipython` está fácil ver una lista de las funciones que provee una biblioteca dada con `math.<TAB>`; en `spyder` ¡está aún más fácil!

Ejercicio: ¿Cuál es la respuesta de la entrada `sqrt(-1)`?

Ejercicio: Resuelve la ecuación cuadrática $ax^2 + bx + c = 0$ para distintos valores de a , b y c . Si “no se puede” entonces dile eso al usuario.

Para permitir operaciones con números complejos como posibles respuestas, se ocupa la biblioteca (*módulo*) `cmath`. Si hacemos

```
import cmath
```

entonces se importan las funciones adecuadas:

```
cmath.sqrt(-1)
```

Si se ocupan las mismas funciones varias veces, o muchas funciones de una sola biblioteca, entonces se vuelve latoso el tener que escribir el `math` todo el tiempo. Hay distintas soluciones para evitar esto.

Una solución es importar la biblioteca con un nombre más corto:

```
import math as m
```

```
m.sin(3.)
m.sqrt(5.)
```

Ahora las funciones que se han importado de `cmath` tienen nombres que empiezan por `cmath.:`

```
cmath.sqrt(-1)
```

`ipython` ahora nos proporciona la lista de funciones adentro del módulo si hacemos `cmath.<TAB>`.

Otra manera es la de importar los nombres deseados al espacio de nombres actual:

```
from math import sin, sqrt
sin(0.5)
sqrt(10.)
cos(0.3)
```

Aquí, la función `cos` dará un error; para poderla utilizar, hay que agregar `cos` a la lista de funciones importadas desde la biblioteca `math`.

Finalmente, hay una manera más natural, pero más peligrosa: se pueden importar *todas* las funciones de la biblioteca:

```
from math import *
sin(0.5)
sqrt(10.)
cos(0.3)
```

El peligro viene cuando uno importa varias bibliotecas: los nombres de funciones provenientes de distintas bibliotecas pueden “colisionarse”, y entonces uno pierde funcionalidad. Uno también duda de cuál biblioteca viene una función dada. Por lo tanto, se suele recomendar *no* utilizar esta forma¹.

Una biblioteca en Python es simplemente un archivo con terminación `.py` que contiene definiciones de funciones. Si creas un tal archivo, se puede usar de inmediato como se detalla arriba, y ¡se vuelve automáticamente una biblioteca! Sin embargo, hay que tener cuidado si hay código ejecutable en el archivo. Para evitar este problema, se puede usar la construcción siguiente:

```
if __name__ == "__main__":
    print "Corriendo el script como programa principal"
else:
    print "Cargando biblioteca"
```

Si se **importa** el archivo, no se correrá la parte del código del **if**.

¹Sin embargo, yo lo utilizo todo el tiempo...

3.5. Argumentos de funciones

Las funciones pueden tener *argumentos por defecto*, que se ocupan si no se da ningún argumento explícito:

```
def saludo(nombre = "David"):
    print "Hola, ", nombre

saludo()
saludo("Albert")
```

Nótese que es obligatorio poner las paréntesis para llamar a la función. Si no, pasa otra cosa:

```
saludo
```

Eso nos permite hacer cosas avanzadas: **Ejercicio:** * Define dos funciones, que regresan el cuadrado y cubo de un número, respectivamente. Dependiendo del valor de una variable, crea un nuevo nombre (“variable”) que es una u otra función. Luego llama a esta función.

Este tipo de argumentos, en donde se especifica el nombre de la variable, se llaman *argumentos de palabra clave* (“keyword arguments”). Si hay varios argumentos de este tipo, podemos especificar uno o ninguno; los que no damos tomarán sus valores por defecto. Además, podemos especificar los argumentos en cualquier orden. Por lo tanto, provee una manera muy flexible de mandar información a una función, y es la manera estándar de que las funciones de las bibliotecas permiten controlar su comportamiento fino.

3.6. ¿Nombres o variables?

En Python, lo que hemos llamado “variables” realmente son nombres de entidades que existen en la memoria. Al poner

```
a = 3
b = a
```

lo que estamos haciendo es crear otro *nombre* `b` del mismo objeto. Eso lo podemos checar con

```
a is b
```

Cada objeto tiene una identificación

```
id(a)
id(b)
```

`is` checa si estas identificaciones son iguales.

Esto tiene implicaciones importantes al usar estructuras de datos más complejos, por ejemplo listas:

```
a = [3, 4, 5]
b = a
b[1] = 17
a[1]
```

Podríamos esperar a que `a[1]` fuera todavía 4, dado que no hemos —aparentemente— modificado `a`. Sin embargo, `a` es un nombre del *mismo objeto* en la memoria que `b`, por lo cual sí se modifica. ¡Ésta es una fuente muy común de errores!

En el caso de listas, se puede realmente hacer una copia, si eso es lo que hace falta, con

```
b = a[:]
b[1] = 17
a[1]
```

3.7. Conversión de números

Acordémonos que hay distintos tipos de números en Python, principalmente enteros (**int**) y flotantes (**float**). A veces, es necesario convertir entre diferentes tipos, incluyendo cadenas, podemos utilizar

```
a = float(3)
b = float('3.5')
```

Para truncar un número flotante a su parte entero, se puede usar simplemente una conversión a entero:

```
a = 3.6
b = int(b)
```

En la biblioteca `math` hay rutinas `floor` y `ceil` para sacar $\lfloor \cdot \rfloor$ y $\lceil \cdot \rceil$ (enteros más cercanos abajo y arriba), así como `round` para redondear. Nótese que estas rutinas regresan flotantes; de requerir enteros, se tienen que convertir explícitamente:

```
a = int(math.round(3.6))
```

Por otro lado, si queremos la representación de un número (o, de hecho, cualquier objeto) como cadena, la podemos obtener con `str`:

```
a = -3.50
str(a)
s = str(a*1j + a)
```

3.8. Aritmética con precisión arbitraria

A veces, es necesaria poder llevar a cabo operaciones aritméticas con números flotantes (reales) con precisión superior a los 16 dígitos que provee el **float** (número de “doble precisión”) de Python. Para hacerlo, existen varios proyectos que proveen bibliotecas con este fin. La mayoría de estas bibliotecas son interfaces a otros proyectos escritos en C++.

Aquí veremos una opción, la biblioteca `mpmath`, que está escrito completamente en Python. En principio eso lo hace más lento, pero más fácil de entender y modificar el código.

Para cargar la biblioteca, hacemos

```
from sympy.mpmath import * #
# from mpmath import * # Tal vez funcione; depende de la instalacion
```

Para cambiar la precisión, hacemos

```
mp.dps = 50
```

Ahora, para crear un número flotante de esta precisión, hacemos

```
x = mpf('1.0')
```

donde el número se expresa como cadena.

Al hacer manipulaciones con `x`, los cálculos se llevan a cabo en precisión múltiple. Por ejemplo,

```
print x/6., x*10
print mpf('2.0')**2**2**2**2**2
```

Con `mpmath`, no hay límite del exponente que se puede manejar. También están definidas muchas funciones, por ejemplo `sin`, `exp` y `log`.

Para imprimir un número con una precisión dada, usamos

```
nprint(x, 20)
```

3.8.1. ¿Cuánto tiempo tarda una función?

Para ver cuánto tiempo ocupa un comando dado de Python, podemos usar `time` o `timeit` en `ipython`:

```
timeit for i in l:\
    print 2*i
```

(El `\` es para indicar que el comando sigue en otra línea.)

Sin embargo, en general, no nos deberíamos preocupar por la velocidad de las operaciones (la optimización) hasta que ya estemos *más que seguros* de que nuestro código funcione perfectamente bien. Como dijo Knuth: *La optimización prematura es la raíz de todos los males.*

3.9. Sustitución de variables en cadenas

Para distintos propósitos, y muy seguido, es útil el poder crear cadenas complejas en las cuales se puedan sustituir valores de variables.

Una manera de hacer esto es con la función `str()`, y concatenar distintas cadenas con `+`:

```
a = 3
s = "El valor de a es " + str(a)
```

Sin embargo, para largas secuencias de datos eso se vuelve latoso.

Una manera más elegante de hacerlo es con la sustitución de variables en cadenas². En una cadena ponemos una secuencia especial de caracteres, empezando por `%`, que indica que se sustituirá el valor de una variable:

```
a = 3
s = "El valor de a es %d"
print s      # cadena valida; todavia no se sustituye
```

Vemos que `s` es una cadena normal de Python, que contiene un `%`.

Ahora aplicamos un operador de Python sobre cadenas, que también se llama `%`, para reemplazar el valor de una variable en lugar del `%d`:

```
a = 3
s = "El valor de a es %d"
s = s % a      # % actua como un operador binario que hace la sustitucion
print s
```

Para reemplazar más de una variable, se usa una tupla:

```
b = 3.5; c = 10.1
s = "b = %g; c = %g" % (b, c)
nombre = "David"
saludo = "Hola %s" % nombre
```

El caracter después del `%` indica el tipo de variable que incluir en la cadena: `d` corresponde a un entero, `g` o `f` a un flotante, y `s` a otra cadena. De hecho, poner `%s` funciona para cualquier tipo:

```
s = "b = %s; c = %s" % (b, c)
```

También se puede poner un número entero, que es el tamaño del campo, y, en el caso de un flotante con `%f`, un punto decimal seguido por un número, que indica el número requerido de decimales:

```
print "%3.7f" % b
```

²Eso parece ser parecido a `printf` en C, pero es más flexible.

3.10. Nueva sintaxis para sustitución de variables en cadenas

La sintaxis discutida arriba para sustituir el valor de una variable en una cadena es la más vieja, aunque aparentemente *no* se removerá de futuras versiones del lenguaje.

Hay una sintaxis nueva que es más flexible. Hay una función (método) `.format` de los objetos tipo cadena, que lleva a cabo el formateo:

```
a = 3
b = 20
c = 10
s = "El valor de a es {0}, de b es {temp}, y de c es {1}".format(a, c, temp=b)
```

Los argumentos numéricos como `{0}` se refieren a los argumentos normales de la función, tomados en orden. Los argumentos con nombres se toman de la lista de argumentos tipo palabra clave.

Los argumentos pueden llevar formatos más complicados:

```
"El valor es {0:8}".format(a)
```

Capítulo 4

Archivos y procesos

Uno de los usos principales de Python para el cómputo científico es el de procesar datos generados en otro lado. Por lo tanto, es necesario poder pasar argumentos hacia un script de Python, importar y exportar archivos, y correr otros programas (*procesos*) desde Python.

4.1. Redirección de la entrada y salida estándar

Si usamos (¡equivocadamente!) un `raw_input` en nuestro programa, podemos mandar la información necesaria al programa usando *redirección* de la *entrada estándar* (por defecto, la terminal) de la forma siguiente:

```
python volumen_esfera.py < radio.dat
```

en donde `radio.dat` contiene la información requerida (aquí, el radio de la esfera). Nótese que esto es una bondad de UNIX, y por lo tanto sólo funciona en Linux y Mac.

De la misma manera, si el programa imprime resultados en la pantalla con `print`, entonces podemos guardar estos resultados en un archivo, en lugar de que salgan a la pantalla:

```
python volumen_esfera.py > volumen.dat
```

Aquí hemos redirigido la *salida estándar*.

Esta forma sobrescribe el contenido del archivo `volumen.dat` con el contenido nuevo; para agregarlo al final del archivo pre-existente, usamos más bien

```
python volumen_esfera.py >> volumen.dat
```

con doble `>>`.

Nótese que esto provee una manera fácil –aunque muy poco flexible– de guardar datos en un archivo. Funciona con cualquier programa (en Linux o Mac) que escribe a la salida estándar (por defecto, la terminal) aún si el programa

4.2. Argumentos de la línea de comandos

Una manera más flexible y más útil para mandar información a un programa es a través de argumentos que se ponen en la mera línea de comandos al momento de mandar correr el programa. A menudo queremos mandar a un programa los valores de un parámetro, por ejemplo, para después poder hacer barridos del mismo.

Para hacerlo, Python provee una variable llamada `argv`, que viene definida en el módulo `sys`. Podemos ver qué es lo que contiene esta variable al correr un programa sencillo:

```
from sys import argv
```

```
print "Argumentos: ", argv
```

Si ahora llamamos a nuestro programa con

```
python prog.py 10 0.5 ising
```

entonces vemos que `argv` es una lista de cadenas, cada una representando uno de los argumentos, empezando por el nombre del script. Por lo tanto, podemos extraer la información deseada con las herramientas normales de Python, por ejemplo

```
from sys import argv
T, h = map(float, argv[1:3])
modelo = argv[3]
```

Sin embargo, si no damos suficientes argumentos, entonces el programa fracasará. Podemos atrapar una *excepción* de este tipo con un bloque `try...except`:

```
from sys import argv, exit
try:
    T, h = map(float, argv[1:3])
    modelo = argv[3]
except:
    print "Sintaxis: python prog.py T h modelo"
    exit(1)
```

Es útil proveerle al usuario información acerca de la razón por la cual fracasó el programa. Aquí hemos utilizado la función `exit` que viene también en el módulo `sys`, para que salga del programa al encontrar un problema, pero eso no es obligatorio –podría poner valores por defecto de los parámetros en este caso, por ejemplo.

Aquí, `map` es una función muy útil, que aplica la función a la cual se refiere su primer argumento a cada elemento de su segundo argumento (que tiene que ser una lista, o, en general, un iterable), y regresa una lista nueva. Nótese que hemos *¿pasado una función como argumento de otra!* Esto suele ser mucho más difícil en otros lenguajes, por ejemplo en C.

4.3. Leer archivos

Para leer un archivo con Python, primero es necesario abrirlo y asociarlo a una variable. Supongamos que tenemos un archivo llamado `datos.dat` (el cual es, por supuesto, un pésimo nombre para un archivo de datos), entonces lo podemos abrir para su lectura con

```
entrada = open("datos.dat", "r")
```

El segundo argumento, `"r"`, indica que se abra para leerse (“read”). El nombre `entrada` ahora denota un objeto de Python que es una representación abstracta del archivo físico en el disco, y que provee métodos (funciones) para interactuar con ello.

Para leer del archivo abierto, hay varias posibilidades. Podemos leer todo de un golpe con `entrada.read()`, leer todo por líneas con `entrada.readlines()`, o línea por línea con `entrada.readline()`. [Nótese que lo que se ha leído ya no se puede leer de nuevo sin cerrar el archivo con `entrada.close()` y volverlo a abrir.]

Por ejemplo:

```
entrada = open("datos.dat", "r")
lineas = entrada.readlines()
lineas
```

nos muestra que `readlines()` regresa una lista de cadenas, siendo cada elemento de la lista una línea del archivo, incluyendo `'\n'` al final, que indica una nueva línea.

Podemos usar esto como sigue:

```
for linea in entrada.readlines():
    print linea
entrada.close()
```

Sin embargo, tal vez la opción más fácil e intuitiva es

```
entrada = open("datos.dat", "r")
for linea in entrada:
    print linea
```

Es decir, el archivo ¡se comporta como una secuencia!, y al iterar, regresa cada vez una nueva línea. Automáticamente toma en cuenta el final del archivo.

Ahora, la línea viene como una sola cadena, con espacios etc. Para extraer la información, primero necesitamos dividirlo en palabras:

```
palabras = linea.split()
```

Si todos los datos en realidad son números, entonces tenemos que procesar cada palabra, convirtiéndola en un número:

```
datos = []
for i in palabras:
    datos.append( float(i) )
```

Resulta que hay una manera más fácil, más intuitivo, de más alto nivel, y, además, más rápida (!) de hacer eso:

```
map(float, palabras)
```

Eso *mapea* literalmente la función `float` sobre la lista `palabras`.

Combinando todo, una manera eficiente y muy pitónica de acceder los datos en un archivo sería:

```
entrada = open("datos.dat", "r")

for linea in entrada:
    datos = map( float, linea.split() )
```

Dentro del bucle podemos manipular los datos como queramos.

[Veremos después que `numpy` provee una manera más fácil y apropiada de leer y escribir tablas de datos, que se debe usar preferentemente para aquel tipo de archivos.]

Ejercicio: Para un archivo con dos columnas, leer los datos de cada columna en dos listas por separado.

Finalmente, podemos usar una comprensión de lista para hacer lo mismo de una manera aún más rápida, intuitiva y bonita:

```
entrada = open("datos.dat", "r")
datos = [ map( float, linea.split() ) for linea in entrada ]
```

Nótese que esto funciona ¡incluso si hay distintos números de datos (columnas) en distintas líneas del archivo!

4.4. Escribir archivos

El método de redirigir la salida que vimos arriba es rápido y fácil. Sin embargo, no es de ninguna manera flexible, por ejemplo no podemos especificar desde nuestro programa el nombre del archivo de salida, ni escribir en dos archivos diferentes.

Para hacer eso, necesitamos poder abrir un archivo para escribirse:

```
salida = open("resultados.dat", "w")
```

La parte más complicada viene al momento de escribir en el archivo. Para hacerlo, ocupamos la función `salida.write()`. Sin embargo, esta función puede escribir solamente *cadena*s. Por lo tanto, si queremos escribir números contenidos en variables, es necesario convertirlos primero a la forma de cadena, como vimos anteriormente. Por ejemplo, podemos hacer una tabla de cuadrados con el código siguiente:

```
salida = open("cuadrados.dat", "w")
for i in range(10):
    salida.write( "%s\t%s\n" % (i, i*i) )
salida.close()
```

[Acordémonos que `\t` denota un tabulador, `\n` denota una nueva línea, y que `%s` se puede utilizar incluso con números.]

4.5. Interacción con otros programas (procesos)

Para poder utilizar Python como “pegamento”, necesitamos poder correr otros programas desde un script de Python, y posiblemente interactuar con ellos (mandarles información, y recibir información de regreso).

El módulo `os` provee unas herramientas sencillas para mandar llamar a otros programas “hijos” desde un programa “padre”. La función más útil es `system`, que permite correr un comando a través de un nuevo `bash`; podemos enviar cualquier comando que funciona en `bash`, en particular podemos correr otros programas. La función acepta una cadena:

```
from os import system
system("ls")
```

Con esto, podemos empezar a hacer cosas interesantes al construir los comandos con la sustitución de variables. Por ejemplo, si tenemos un programa `prog.py` que acepta un argumento de la línea de comandos, podemos correrlo de manera consecutiva con distintos valores del parámetro con

```
for T in range(10):
    comando = "python prog.py %g" % T
    system(comando)
```

Sin embargo, no provee realmente la manera de interactuar con el programa. Para esto, hoy día se utiliza el paquete `subprocess`, que es muy flexible.

La manera más fácil de utilizar `subprocess` es llamar a un programa externo y checa si terminó bien. [Los programas regresan un número que indica esto. En Python, el número se puede dar como argumento de `sys.exit()`.]

```
import subprocess
comando = "ls"
exito = subprocess.call(comando)
```

El valor será 0 si el proceso terminó bien. [Lo mismo se puede hacer con `os.system()`.]

Las opciones del comando se pueden pasar como otros argumentos de `call()`, metidos en una lista de cadenas. Una versión alternativa, `check_call()`, produce una excepción si hay

```
comando = 'ls'
exito = subprocess.call([comando, '-l'])
```

Para recoger la salida del comando (que normalmente va a la salida estándar, es decir, la terminal), se utiliza `check_output()`:


```
salida_ls = subprocess.check_output(['ls'])
```

Para las tareas más avanzadas, existe un objeto `subprocess.Popen`, que permite interactuar con el proceso, por ejemplo, para enviar comandos a `gnuplot`.

Parte II

Paquetes para el cómputo científico

Capítulo 5

La biblioteca `numpy` para vectores y matrices

La biblioteca `numpy` es fundamental para el cómputo científico de cualquier tipo con Python, ya que provee objetos para manipular de manera intuitiva- y eficientemente vectores y matrices matemáticos.

5.1. Necesidad de una biblioteca especializada para cómputo científico

Hasta ahora, hemos utilizado listas para guardar y manipular datos. Sin embargo, las listas no se comportan como vectores, y menos como matrices –al sumarlos no se comportan de la manera adecuada, etc, por ejemplo

```
1 = [3, 4, 5]
1 + [1, 2, 3]
2 * 1
```

El propósito de la biblioteca `numpy` es justamente el de proporcionar objetos que sí representan a vectores y matrices matemáticos, con todas las bondades que traen consigo este tipo de objetos.

La manera usual de cargar la biblioteca es con

```
import numpy as np
```

(Acordémonos que esto carga la biblioteca con nombre `np` en lugar de `numpy`, para poder teclear menos.) Provee muchas funciones para trabajar con vectores y matrices.

[Se suele usar también

```
from numpy import *
```

Sin embargo, esto se desaconseja, ya que introduce muchos nombres al espacio de nombres global que podrían sobrescribir nombres provenientes de otros paquetes o de variables definidas por el usuario. El cargar la biblioteca así mantiene todos los nombres dentro del nombre `np` del módulo.]

Los vectores se llaman (aunque es un poco confuso) `arrays`, y se pueden crear de distintas maneras. Tienen algunas de las propiedades de las listas, pero con propiedades y métodos adicionales para funcionar como objetos matemáticos. La manera más general de crear un vector es el convertir una lista a un `array`, que se hace con el *constructor* (función que construye objetos) también llamada `array`. Se pasa una lista a esta función y regresa un objeto de tipo `array` que funciona como vector matemático.

```
a = np.array( [1, 2, -1, 100] )
```

Se tiene que tomar cuidado con el orden de las paréntesis y los corchetes. Acordémonos que las paréntesis son para indicar los argumentos de una función, y a esta función se pasa una lista, la cual se delimita con corchetes.

Un vector de este tipo puede contener sólo un tipo de objetos, a diferencia de una lista normal de Python; esto se debe a la necesidad de eficiencia. Si todos los números en la lista son enteros, entonces el tipo del arreglo

también lo es, como se puede comprobar con `a.dtype`.

Si creamos muchos arrays a mano, puede convenir no tener que teclear `np.array` cada vez. Hay dos soluciones para usar sólo `array` en su lugar:

```
from numpy import array
```

o

```
array = np.array
array          # regresa el objeto al cual se refiere el nombre 'array'
```

Los introducen el sinónimo (otro nombre) `array` para el objeto `numpy.array`.

Si hacemos

```
a = array( [1, 2, 3] )
type(a)
```

vemos que `a` es un objeto de tipo `numpy.ndarray`, es decir un *arreglo n -dimensional*. Veremos que las matrices, que tienen dos índices (es decir, son de dimensión 2), también son arrays, y de igual forma se pueden crear objetos con un número cualquiera de índices. El nombre `array` se refiere, entonces, a cualquier objeto de este tipo (arreglo con n índices, para cualquier valor (entero positivo) de n).

5.2. Operando con vectores

Los vectores creados de esta manera se pueden sumar, restar etc., y se comportan como vectores matemáticos. Todas las operaciones se llevan a cabo entrada por entrada:

```
a = np.array( [1., 4., 7. ] )
b = np.array( [1., 2., -2. ] )
print a+b, a-b, a*b, a/b, a**b
```

Ejercicio: ¿Cómo se puede crear un vector de 100 copias del número -3 ?

Las funciones más comunes entre vectores ya están definidas en `numpy`, entre las cuales se encuentran `dot(a,b)` para productos escalares de dos vectores de la misma longitud, y `cross(a,b)` para el producto cruz de dos vectores dos- o tres-dimensionales (es decir, con tres entradas cada uno).

5.3. Extrayendo partes de un vector

La misma sintaxis funciona para vectores como para listas para extraer partes: las componentes (entradas) individuales se extraen con `a[i]`:

```
a = np.r_[0, 1, 2, 3]
print a[0], a[2]
```

y los “subvectores” [rebanadas (“slices”)] con

```
b = a[1:3]
```

Nótese, sin embargo, que en este caso la variable `b` *no* es una *copia* de esta parte de `a`. Más bien, es una *vista* de `a`, así que ahora si hacemos

```
b[1] = 10
```

entonces la entrada correspondiente de `a` ¡*también cambia!* [Este fenómeno también pasa con listas:

```
l=[1,2,3]; k=l; k[1] = 10
```

```
]
```

5.4. Creando vectores de forma rápida

A menudo queremos crear vectores de cierto tamaño con todos ceros o unos:

```
b = np.zeros(10)
b = np.ones(10)
```

También hay distintas maneras de crear vectores que consisten en rangos ordenados, por ejemplo `arange`, que funciona como `range`, con un punto inicial, un punto final, y un paso:

```
a = np.arange(0., 10., 0.1)
```

y `linspace`, donde se especifica puntos iniciales y finales y un número de entradas:

```
l = np.linspace(0., 10., 11)
```

Una notación abreviada para construir vectores es `r_`, que se puede pensar como una abreviación de “vector renglón”:

```
a = np.r_[1, 2, 10, -1.]
```

Este método se extiende para dar una manera rápida de construir rangos:

```
np.r_[3:7]
np.r_[3:7:0.5]
np.r_[3:7:10j]
```

Este último utiliza un “número complejo” simplemente como otra notación, y es el equivalente de `linspace(3, 7, 10)`.

5.5. Funciones de vectores

`numpy` contiene muchas funciones matemáticas que están hechas para operar tanto sobre números, como sobre vectores:

```
a = np.r_[1, 2, 3.]
b = 17
np.sin(a)
np.sin(b)
type(np.sin)      # 'ufunc'
```

El tipo de estas funciones es `ufunc`, que quiere decir “función universal”. Operan sobre `ndarrays` elemento por elemento, regresando un `array` nuevo con los resultados; se puede considerar una versión *vectorizada* de la función equivalente escalar.

Es más: al definir una función el usuario, esta función a menudo también se pueden aplicar directamente a un vector:

```
def f(x):
    return np.sin(x) + x*x
```

Dada la manera en la que trabaja Python, si pasamos a esta función un número, regresará un número. Pero si le pasamos un `array`, ¡regresará el `array` correspondiente!

Ejercicio: Escribe una función de una variable que regresa la gaussiana $gauss(x) = \frac{1}{\sqrt{2}} \exp[-x^2/2.]$, tal que sirva de manera igual para un número o un arreglo de `numpy`.

5.6. Matrices

Las matrices se tratan como vectores de vectores, o listas de listas:

```
M = np.array( [ [1, 2], [3, 4] ] )
```

La forma de la matriz se puede ver con

```
M.shape
```

y se puede manipular con

```
M.shape = (4, 1); print M
```

o con

```
M.reshape(2, 2)
M
```

Nótese que en este caso, el objeto `M` no cambia; más bien, se regresa una vista de `M`. De hecho, esto provee una manera útil de crear las matrices:

```
M = r_[0:4].reshape(2, 2)
print M
```

Para más generalidad, podemos crear una matriz desde una función:

```
def f(i, j):
    return i+j

M = fromfunction(f, (3, 3))
M
```

Dado que las matrices son vectores de vectores, al hacer

```
print M[0]
```

nos regresa la primera componente de `M`, que es justamente un *vector*, el primer renglón de `M`. Si queremos cierta entrada de la matriz, entonces más bien necesitamos especificar dos coordenadas:

```
M[0][1]
M[0, 1]
```

Para extraer ciertas renglones o columnas de `M`, utilizamos una extensión de la notación para vectores:

```
M = identity(10) # matriz identidad de 10x10
M[3:5]
M[:, 3:5]
M[3:9, 3:5]
```

Podemos calcular la transpuesta de la matriz `M` con

```
M.transpose()
```

o más brevemente con

```
M.T
```

Pero nótese que estas funciones *no* modifican `M`, si no son *vistas*. Por lo tanto, si ponemos


```
Z = M.T
Z[1,1] = 17
```

entonces *también* se modifica la entrada correspondiente de `M`. En este caso, y en el caso de `reshape()`, lo único que está pasando es que se cambian los *strides*, disponibles con `M.strides()`, que indican los desplazamientos necesarios para acceder el siguiente elemento de `M` en la dirección dada. [Internamente, los datos de la matriz se guardan simplemente como un arreglo uni-dimensional de números.]

Para evitar este comportamiento, es necesario forzar a Python que haga más bien una *copia* nueva de la matriz:

```
Z = M.copy()
Z[1,1] = 17
```

Una función que puede ser útil para construir matrices con entradas repetidas es `tile`

```
tile(M, (2,2) )
```

Otros métodos útiles son `diagonal`, que regresa una diagonal de un arreglo:

```
diagonal(M)
diagonal(M, 1)
```

y `diag`, que construye una matriz con el vector dado como diagonal:

```
diag([1,2,3])
diag([1,2,3], 2)
```

5.7. Broadcasting

¿Qué pasa al hacer lo siguiente?:

```
v = np.array( [1, 2, 3] )
w = v + 1
```

Estamos intentando sumar un número a un vector. En principio, esto no debería funcionar, ya que los objetos son de distintos tipos. Sin embargo, Python hace algo muy razonable: suma 1 a *cada elemento* de `v`.

Lo mismo pasa con una matriz:

```
M = np.array( [1, 2, 3, 4] ).reshape(2, 2)
N = M + 1
```

De hecho, también funciona lo siguiente:

```
M = np.array( [1, 2, 3, 4] ).reshape(2, 2)
N = M + np.array( [5, 6] )
```

Aquí, estamos sumando un vector de longitud 2 a una matriz 2×2 . Python “broadcast-ea” (“difunde”) el vector sobre la matriz, es decir, que intenta hacer algo razonable. Sin embargo, no siempre se puede, si los tamaños no son adecuados:

```
M = np.array( [1, 2, 3, 4] ).reshape(2, 2)
N = M + np.array( [5, 6, 7] ) # error!
```

Aunque el broadcasting puede ser útil, también es algo opaco para un lector de tu código (por ejemplo, para ti dentro de una semana, un mes, ¡o un año! Es más, puede llevar a errores difíciles de encontrar, ya que uno piensa estar sumando dos vectores, pero en realidad, por ejemplo, se le olvidó a uno multiplicar un escalar por un vector, y está sumando un vector y un escalar. En lugar de darte un error, Python lo trata equivocadamente como un caso de broadcasting¹.

¹¡Éste es un caso real que le pasó a un estudiante mío recientemente!

5.8. Comparación de arrays

Dos arrays se pueden comparar elemento por elemento:

```
a = r_[1, 2, 3]
b = r_[4, 1, 5]
a < b
```

La comparación regresa un array de **True** (verdadero) y **False** (falso), es decir, entradas booleanas.

Podemos checar si todos alguna de las comparaciones individuales sea cierta con `np.any()`, o si todas lo sean, con `np.all()`:

```
any(a < b)
all(a < b)
```

También podemos seleccionar a los elementos de un arreglo que satisfacen cierta propiedad:

```
a > 5
b = a[a > 5]      # escoge subconjunto de las entradas
a[a > 5] = 0      # pone las entradas correspondiente a 0 (usando broadcasting!)
```

Además, existe una función `where`, que se comporta como una versión vectorizada de `if`:

```
b = r_[0:16].reshape(4,4)
c = list(b.flatten())
c.reverse()
c = array(c).reshape(4,4)

a = where(b < 5, b, c)
```

Este comando pone cada entrada de `a` igual a la entrada correspondiente de `b` si ésta es menor que 5, o a la de `c` si no.

5.9. Funciones para resumir información sobre vectores y matrices

Hay varias funciones que regresan información resumida sobre vectores y matrices, por ejemplo

```
a = r_[0:16].reshape(4,4)

a.max()
a.min(1)      # actua sobre solamente un eje y regresa un vector
a.mean(0)
a.mean(1)
a.sum(1)
```

5.10. Multiplicación de matrices y vectores

Si tenemos una matriz y un vector:

```
M = r_[1, 2, 3, 4].reshape(2, 2)
v = r_[5, 6]
```

podríamos querer hacer el producto matriz–vector, así como el producto matriz–matriz.

Sería natural pensar en utilizar el operador `*` para este fin, pero acordémonos que este operador ya está tomado para otra función (multiplicación elemento por elemento; en el caso de $M \star v$, ocupa broadcasting).

Estos dos tipos de producto de matrices corresponden, en términos matemáticos, a una *contracción de índices*, es decir, una suma sobre un índice mudo:

$$(\mathbf{M} \cdot \vec{v})_i = \sum_j M_{ij} v_j; \quad (5.1)$$

$$(\mathbf{M} \cdot \mathbf{N})_{ij} = \sum_k M_{ik} N_{kj}. \quad (5.2)$$

Los dos se pueden considerar, entonces, generalizaciones del producto punto de dos vectores; por lo tanto, en Python se utiliza la misma sintaxis, con `dot`:

```
dot(M, v)
dot(M, M)
```

Ejercicio: Utiliza el *método de potencias* para calcular el vector propio de una matriz cuadrada M correspondiente al valor propio de mayor módulo. Este método consiste en considerar la iteración $M^n \cdot v$, renormalizando el vector que se produce en cada vuelta de la iteración.

Ejercicio: ** Extender el método de potencias para calcular más de un eigenvector, en el caso de matrices simétricas, para las cuales se sabe que los eigenvectores son *ortogonales*.

5.11. Leer un array desde un archivo

Para leer un archivo que contiene datos para guardarse en un `array`, podemos usar `np.loadtxt`:

```
a = np.loadtxt("a.dat")
```

Podemos cargar solamente ciertas columnas del archivo al arreglo y saltar cierto número de líneas, usando los argumentos adicionales de la función.

Ejercicio: Usa manipulaciones de `numpy` para cargar un archivo de datos que consiste en 2 columnas, y repartir las dos columnas entre dos variables `x` y `y`, en una sola línea.

También existe otra función llamada `np.genfromtxt`. Esta función permite construir un arreglo desde una cadena de texto arbitraria, y tiene más flexibilidad para manejar datos faltantes.

5.12. Escribir un array a un archivo

Para escribir el contenido de un `array` a un archivo, podemos usar `np.savetxt`:

```
import numpy as np
a = np.identity(5)                # matriz identidad
np.savetxt("a.dat", a)
```

Al consultar el manual de `saveetxt`, vemos que hay otras opciones que podemos especificar al poner palabras clave, e.g.

```
np.savetxt("a.dat", a, fmt="%g", delimiter="\t")           # fmt da el formato
```


Capítulo 6

Submódulos de numpy para cómputo científico: álgebra lineal, transformadas de Fourier y números aleatorios

`numpy` está diseñado principalmente para proporcionar el tipo `ndarray` para representar vectores y matrices de una manera muy eficiente, y las manipulaciones correspondientes. Sin embargo, también contiene tres submódulos para llevar a cabo tareas fundamentales para el cómputo científico: álgebra lineal, transformadas (rápidas) de Fourier, y números aleatorios. [Por lo tanto, `numpy` se puede considerar un *paquete*, ya que empaqueta varios submódulos.]

Los primeros dos submódulos proveen interfaces fáciles de utilizar a rutinas clásicas (LAPACK, y FFTW (?), respectivamente) escritas en Fortran (con un interfaz horrible...); hay versiones más completas de los dos dentro del paquete `scipy` [ver el capítulo correspondiente].

6.1. Álgebra lineal

Dentro de `numpy` hay un (sub-)módulo `linalg` que permite llevar a cabo las operaciones más frecuentes de álgebra lineal numérica.

```
from numpy import linalg
```

Como siempre, para averiguar cuáles funciones hay adentro del módulo, en `ipython` hacemos, como siempre, `linalg.<TAB>`, y para conseguir ayuda con una función dada se hace, por ejemplo, `linalg.norm?`. Muchas funciones incluso tienen su código fuente disponible: `linalg.norm??`. [Nótese que si ocupas una terminal con un color de fondo ligero, podría ser útil usar el comando mágico de IPython `%colorsLightBG` para que los colores aparecen adecuados.]

Las funciones se usan como sigue:

```
linalg.norm(v)           # norma de un vector
```

Una alternativa es importar todas las funciones de este submódulo

```
from numpy.linalg import *
```

entonces ya no se pone `linalg.`, y se puede referir simplemente a `norm`.

Algunas de las funciones útiles incluyen `linalg.eig` para calcular los valores y vectores propios de una matriz:

```
linalg.eig(M)
```

Esta función regresa un par ordenado (valores, vectores), así que podemos poner

```
lamb, v = linalg.eig(M)
```

donde `lamb` contendrá los valores propios, y `v` los vectores propios. [La palabra `lambda` es una palabra reservada en Python, por lo cual no se puede utilizar como nombre de una variable.] Cabe enfatizar que los vectores propios se regresan como las *columnas* de la matriz.

`linalg.eigvals` regresa solamente los valores propios. Así que podemos poner por ejemplo

```
lamb, v = linalg.eig(M)
```

[Nótese que `lambda` es una palabra reservada que no se puede modificar.]

Hay versiones especiales `linalg.eigh` y `linalg.eigvalsh` para matrices hermitianas o simétricas reales.

También hay funciones `linalg.det` para determinante, e `linalg.inv` para la inversa de una matriz.

Finalmente, una función muy importante es `linalg.solve()` para resolver un sistema de ecuaciones lineales $M \cdot x = b$:

```
x = linalg.solve(M, b)
```

Para averiguar exactamente cuál rutina se utiliza, podemos ver el código fuente en Ipython:

```
linalg.solve??
```

6.2. Transformadas de Fourier

El submódulo `numpy.fft` provee transformadas rápidas de Fourier:

```
import numpy.fft as fft # o: from numpy import fft
x = arange(1024)
f = fft.fft(x)
y = fft.ifft(f)
linalg.norm(x - y)
```

6.3. Números aleatorios

Nótese que en general, *no es posible* generar números verdaderamente aleatorios en una computadora. Más bien son números *seudo-aleatorios*, que se generan con una iteración determinista, pero que produce una salida que “se ve altamente aleatorio”.

Las funciones para generar números (seudo-)aleatorios que provee `numpy` están en el submódulo `numpy.random`.

Como siempre, las funciones se llaman, por ejemplo, `random.random()`. Para facilitarnos la vida, podemos importar todas estas funciones al espacio de nombres con

```
from numpy import random
random? # informacion sobre el modulo
```

[Nótese que hay otro módulo `random` que existe afuera de `numpy`, con distinta funcionalidad; para fines científicos, la versión dentro de `numpy` es la adecuada.]

La funcionalidad básica del módulo es la de generar números aleatorios distribuidos de manera uniforme en el intervalo $[0, 1)$:

```
random.random() # la funcion 'random' adentro del modulo numpy.random
for i in xrange(10):
    print random.random()
```

Podemos generar vectores y matrices con entradas aleatorias:

Para generar una matriz aleatoria, podemos utilizar

```
v = random.rand(10)
M = random.rand(20, 20)
d = np.dot(M, v)
```

Para números en un cierto rango $[a, b)$, podemos utilizar

```
random.uniform(-5, 5, 10)
```

También hay diversas funciones para generar números aleatorios con distribuciones no-uniformes, por ejemplo `random.exponential(10)` (distribución exponencial), `random.randn(10, 5, 10)` (distribución normal o gaussiana), y `random.binomial(10, 3, 100)` (distribución binomial).

Ejercicio: Genera una matriz real y simétrica de 1000×1000 . Calcula numéricamente la distribución (histograma) de las distancias entre sus valores propios consecutivos. [En el siguiente capítulo, veremos cómo graficar el resultado.]

Capítulo 7

Gráficas 2D con matplotlib

En este capítulo, veremos cómo podemos crear gráficas con calidad de publicación desde Python, usando el paquete `matplotlib` y su entorno `pylab`.

`matplotlib` es un paquete bastante completo para producir gráficas (y, hasta cierto punto, animaciones), principalmente en 2D. Puede guardar el resultado en varios formatos, incluyendo en particular PDF para incorporar en publicaciones.

7.1. Entorno `pylab`

El paquete `matplotlib` dibuja las gráficas. Para integrar mejor este módulo con el uso interactivo, y en particular con `ipython`, se incluye un módulo `pylab`, que provee muchos comandos de utilidad para manejar `matplotlib`, y que está basado en la sintaxis de Matlab.

La manera más efectiva de cargar la biblioteca `pylab` es al mero momento de *correr* `ipython`: se da la opción `--pylab` en la línea de comandos¹:

```
ipython --pylab
```

Si todo funciona correctamente, deberías recibir el mensaje “Welcome to pylab, a matplotlib-based Python environment.” En este caso, ya se habrá cargado el entorno `pylab`, que incluye el módulo `matplotlib` para llevar a cabo gráficas, y también carga automáticamente `numpy`, por lo cual no es necesario volver a cargar estos módulos para el uso interactivo. [Sí es necesario cargarlos explícitamente en cualquier script que ocupe gráficas.]

7.2. Gráficas básicas

El comando principal de `matplotlib` / `pylab` es `plot()`. Acepta uno o dos listas o vectores de `numpy`, que corresponden a las coordenadas y (si hay un solo vector) o x y y de una gráfica 2D. Por ejemplo, si queremos graficar los cuadrados de los números de 1 a 10, podemos hacer

```
from matplotlib import pyplot
from numpy import arange
x = arange(10)
y = x * x
pyplot.plot(x, y)
```

Si hacemos esto desde la consola de `Ipypthon`, podemos poner simplemente

¹Es conveniente declarar un nuevo comando `pylab` agregando la línea `alias pylab='ipython --pylab'` en el archivo `.bashrc` en mi directorio hogar. Entonces se puede correr poniendo el comando `pylab` en la línea de comandos. Al instalar la Enthought Python Distribution en Mac, viene un programa `pylab` ya hecho.

```
x = arange(10)
y = x * x
plot(x, y)
```

Para obtener el mismo efecto desde un script, podemos utilizar

```
from pylab import *
```

Sin embargo, esto ¡no se recomienda!

Si queremos puntos en lugar de líneas, hacemos

```
plot(x, y, 'o')
```

al estilo de Matlab. Si se utiliza la versión `ipython-pylab`, entonces la figura se debería desplegar automáticamente.

Nótese que por defecto las gráficas se *acumulan*. Este comportamiento se puede modificar con `hold(False)`, que reemplaza las gráficas con las nuevas, y `hold(True)`, que regresa a la funcionalidad por defecto. También se puede utilizar `clf()` para limpiar la figura.

La ventana que cree `matplotlib` incluye botones para poder hacer acercamientos y moverse a través de la gráfica. También incluye un botón para exportar la figura a un archivo en distintos formatos. Los formatos de principal interés son PDF, que es un formato “vectorial” (que incluye las instrucciones para dibujar la figura), que da la calidad necesaria para las publicaciones, y PNG, que da una imagen de la figura, y es adecuada para páginas web, o para figuras con demasiados puntos o símbolos. Los otros formatos básicamente ya no se usan.

Las gráficas también se pueden guardar con un comando:

```
savefig("grafica.pdf")
savefig("grafica.png")
```

El formato del archivo está determinado automáticamente por la terminación del nombre.

7.3. Cambiando el formato

Hay distintos tipos de líneas y puntos disponibles, y se puede modificar el tamaño de ambos. Todas las opciones están disponible a través de la documentación de `plot`, a través de `plot?`. Además, los colores se pueden especificar explícitamente:

```
x = arange(10)
plot(x, x**2, 'ro', x, 2*x, 'gx')
plot(x, 3*x, 'bo-', linewidth=3, markersize=5, \
      markerfacecolor='red', markeredgecolor='green', markeredgewidth=2)
```

Aquí, se utilizan argumentos de palabra clave para especificar las opciones; cada opción tiene un valor por defecto.

7.4. Etiquetas

Se puede proporcionar un título de la gráfica con `title`

```
title("Algunas funciones sencillas")
```

Los ejes se pueden etiquetar con

```
xlabel('x')
ylabel('f(x)')
```

Una bondad de `matplotlib` es que las etiquetas se pueden expresar en formato $\text{\LaTeX}$ y se interpretará automáticamente de la forma adecuada:

```
xlabel('$x$')
ylabel('$x^2$,  $2x$ ,  $\exp(x)$ ', fontsize=16)      # cambia tamaño de fuente
```

Esto utiliza una versión interna de $\text{\TeX}$ y por lo tanto sólo acepta una gama limitada de comandos. Se puede utilizar $\text{\TeX}$ de verdad con un cambio en la configuración de `Matplotlib`, en cuyo caso se puede aprovechar la gama entera de comandos de $\text{\LaTeX}$.

También se pueden colocar etiquetas en lugares arbitrarios con

```
text(4.6, 35, "Punto de interés")
```

Si se le asigna a una variable, entonces se puede volver a remover con

```
etiq = text(4.6, 35, "Punto de interés")
etiq.remove()
draw()
```

Es necesario llamar a `draw()` para volver a dibujar la figura.

También es posible hacer una anotación con una etiqueta combinada con una flecha, con `pyplot.annotate()`:

```
x = linspace(-1, 1, 100)
y = x**3
plot(x, y)

annotate('inflección', xy=(0, 0), xytext=(0.5, 0.5),
         arrowprops=dict(facecolor='black', shrink=0.1) )
```

Aquí, `dict()` crea un diccionario con las entradas dadas, que es el tipo requerido para especificar las propiedades de cómo aparece la flecha, en `arrowprops`. (Sin `arrowprops`, no aparece ninguna flecha.)

7.5. Figuras desde scripts

Una vez que una figura se vuelve complicada, es necesario recurrir a un script que contenga las instrucciones para dibujarla. Eso es *sumamente importante* en particular para preparar figuras para un documento, donde se ajustará varias veces una sola figura hasta que quede bien.

Para utilizar `matplotlib` desde un script, es necesario incluir la biblioteca `pylab`. Luego se puede utilizar `plot`. Para ver la imagen, a veces es necesario poner el comando `show()`:

```
from matplotlib.pyplot import plot, show
from numpy import arange
x = arange(10)
plot(x, x**2)
show()
```

Para el uso interactivo, el usar `pylab` es la manera más fácil de cargar la librería. Todos los nombres de las funciones y variables en `numpy` y `matplotlib.pyplot` están entonces disponibles. Para programar, es más conveniente mantener los nombres separados:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 10, 0.1)
y = np.cos(x)
p = plt.plot(x, y)
```

En este caso se creó la figura como el objeto `p`, pero no se despliega. Se puede, por ejemplo, ahora guardar la figura con `savefig` para el uso en un script. Si se quiere desplegar la figura en la pantalla, se puede hacer con

```
plt.show()
```

Nótese que en este caso será necesario cerrar la figura para poder seguir en `ipython`.

Para cambiar el rango de los ejes, podemos usar `axis`:

```
axis([-3, 3, -3, 3])
```

El orden de los argumentos es los mínimos y máximos en la dirección x , y luego lo mismo en la dirección y .

7.6. Escalas logarítmicas

Para utilizar ejes con escalas logarítmicas, hay tres funciones: `loglog`, `semilogy` y `semilogx`, las cuales se ocupan en lugar de `plot`:

```
from matplotlib.pyplot import loglog
t = arange(1000)
p = t**(-0.5)
loglog(t, p, 'o')
```

7.7. Múltiples dibujos (subplot)

Múltiples subdibujos alineados se pueden crear con `subplot`. Su sintaxis es

```
subplot(num_renglones, num_columnas, num_dibujo)
```

Es decir, se especifican el número de renglones y columnas que uno quiere, y el último número especifica cuál dibujo es los `num_renglones × num_columnas` es. Aquí está un ejemplo adaptado de la documentación de `matplotlib`:

```
def f(t):
    """Oscilacion amortiguada"""
    c = cos(2*pi*t)
    e = exp(-t)
    return c*e

t1 = arange(0.0, 5.0, 0.1)
t2 = arange(0.0, 5.0, 0.02)
t3 = arange(0.0, 2.0, 0.01)

subplot(211)
l = plot(t1, f(t1), 'bo', t2, f(t2), 'k--', markerfacecolor='green')
grid(True)
title('Amortiguacion de oscilaciones')
ylabel('Amortiguada')

subplot(212)
plot(t3, cos(2*pi*t3), 'r.')
grid(True)
xlabel('tiempo $t$ (s)')
ylabel('No amortiguada')
show()
```

7.8. Otros tipos de gráficas

Aparte del tipo básico de gráficas que hemos visto hasta ahora, que permiten llevar a cabo gráficas de datos como puntos y/o líneas, existe en `matplotlib` una gran variedad de otros tipos de gráficas posibles; se refiere a la página principal <http://matplotlib.sourceforge.net> y a la página <http://www.scipy.org/Cookbook/Matplotlib> que contiene muchos ejemplos.

Por ejemplo, podemos visualizar matrices 2D como “heat-maps” (mapas en los cuales el color corresponde al valor de la matriz) con `pcolor` (“pseudo-color”) e `imshow`. Nótese que para crear una figura nueva con forma cuadrada, se puede utilizar

```
f = figure( figsize=(8,8) )
```

Otro tipo de gráfica disponible es un histograma. `pylab` puede calcular y dibujar la gráfica en un solo comando:

```
from pylab import randn, hist
x = randn(10000)
hist(x, 100, normed=True)
```

Hay una función `histogram` en `numpy` que calcula histogramas sin dibujarlos.

Aquí va un ejemplo tomado de la documentación:

```
import numpy as np
import matplotlib.mlab as mlab
import matplotlib.pyplot as plt

mu, sigma = 100, 15
x = mu + sigma*np.random.randn(10000)

# the histogram of the data
n, bins, patches = plt.hist(x, 50, normed=1, facecolor='green', alpha=0.75)

# add a 'best fit' line
y = mlab.normpdf( bins, mu, sigma)
l = plt.plot(bins, y, 'r--', linewidth=1)

plt.xlabel('Smarts')
plt.ylabel('Probability')
plt.title(r'$\mathrm{Histogram\ of\ IQ:}\ \mu=100,\ \sigma=15$')
plt.axis([40, 160, 0, 0.03])
plt.grid(True)

plt.show()
```

Ejercicio: Genera una matriz aleatoria real. Encuentra sus valores propios y dibújalos

Ejercicio: Genera una matriz aleatoria real y simétrica. Encuentra sus valores propios y dibújalos. Encuentra las diferencias entre valores propios consecutivos (en tamaño), y dibújalas. Encuentra su distribución y dibújala.

****** Compárala con el resultado exacto en el caso cuando las entradas de la matriz son gaussianas.

Capítulo 8

Matplotlib avanzado

8.1. Matplotlib para dibujar figuras

Aparte de gráficas tal cual, podemos utilizar `matplotlib` para dibujar figuras que representan objetos matemáticos, simplemente juntando distintos tipos de rectas, círculos etc.

Para agregar objetos más complicados, como un círculo, a un `plot` de `matplotlib`, es necesario entrar en la estructura subyacente orientada a objetos de `matplotlib`.

En particular, tiene un concepto de *patches* (“patches”), que son objetos rellenos que se pueden agregar al dibujo.

```
import numpy as np
import matplotlib.pyplot as plt

circ = plt.Circle((0.5, 0.5), radius=0.1, color='b')

ax = plt.gca()
ax.add_patch(circ)

plt.draw()                # redibujar
```

La función `gca()` regresa el conjunto de ejes actual. `Circle()` crea un *parche*, el cual se tiene que agregar al `plot` con `add_patch`.

Ejercicio: Genera una matriz aleatoria de 0s y 1s. Dibuja una representación geométrica de la matriz como un arreglo cuadrado, con círculos azules en lugar de 1s y cuadrados rojos en lugar de 0s. Dibuja una malla con líneas punteadas para indicar la ubicación de cada celda.

Hay varios tipos de objetos que funcionan de la misma forma, por ejemplo flechas:

```
import numpy as np
import matplotlib.pyplot as plt

x = arange(10)
y = x

plot(x, y)
arr = plot.Arrow(2, 2, 1, 1, edgecolor='white')
ax = plot.gca()    # obtener el objeto tipo "eje" ("axis")
ax.add_patch(arr)
arr.set_facecolor('g')
show()
```

Los parches se pueden rotar:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import matplotlib as mpl

fig = plt.figure()
ax = fig.add_subplot(111)

r1 = patches.Rectangle((0,0), 20, 40, color="blue", alpha=0.50)
r2 = patches.Rectangle((0,0), 20, 40, color="red", alpha=0.50)

t2 = mpl.transforms.Affine2D().rotate_deg(-45) + ax.transData
r2.set_transform(t2)

ax.add_patch(r1)
ax.add_patch(r2)

plt.xlim(-20, 60)
plt.ylim(-20, 60)

plt.grid(True)

plt.show()
```

(Ejemplo tomado de un grupo en línea.)

La mayoría de los tipos de parches se definen en el submódulo `matplotlib.patches`, así que para ver cuáles hay, podemos hacer

```
import matplotlib.patches
dir(matplotlib.patches)
matplotlib.patches.<TAB> # alternativa para IPython
```

8.2. Campos escalares y vectoriales: Mallas con meshgrid

Un campo escalar es una función que le asigna un número a cada punto del espacio. De igual forma, un campo escalar le asigna un vector a cada punto del espacio.

Para crear representaciones gráficas de campos escalares y vectoriales en 2D, podemos especificar el valor escalar o vectorial en un cierto conjunto de puntos del espacio.

Por lo tanto, es necesario primero construir la malla donde se calculará el campo vectorial. Esto se hace con `meshgrid`, una función que acepta dos vectores 1D que dan las coordenadas x y y de los puntos de la malla. `meshgrid` construye una malla cuyos vértices son los puntos en el producto cartesiano de los dos conjuntos de puntos. Regresa una tupla de dos `arrays`, que son matrices que contienen las coordenadas x y y , respectivamente, de todos los puntos de la malla:

```
import numpy as np
x = np.r_[-5: 5: 11j] # 11 entradas
y = np.r_[-5: 5: 11j]
X, Y = np.meshgrid(x, y)
X
Y
np.meshgrid?? # el codigo de meshgrid es interesante!
```


Por lo tanto, los resultados de `meshgrid` proveen una manera de mear las entradas de las dos matrices x y y a las coordenadas correspondientes de los puntos de la malla en el espacio 2D.

El punto de esta construcción aparentemente complicada es que ahora las funciones se pueden evaluar en *cada punto de la red al mismo tiempo*. Por ejemplo, para tener una matriz que representa la suma de las componentes x y y en cada punto de la malla, hacemos simplemente

```
Z = X + Y
```

Para tener la distancia de la origen en cada punto, hacemos

```
R = np.sqrt(X*X + Y*Y)
```

Estas operaciones funcionan componente por componente. Dado que las matrices x y y contienen el mismo número de entradas, y que sus componentes correspondientes se refieren a un mismo punto, provee una manera sumamente sucinta y eficiente de calcular funciones de este tipo.

Ahora lo podemos como “heat-map” o con contornos como

```
from matplotlib import pyplot as plt
plt.pcolor(X, Y, R)
plt.pcolormesh(X, Y, R) # mas rapido para arreglos grandes; se prefiere

plt.contour(X, Y, R)
plt.contourf(X, Y, R) # rellena regiones entre contornos
```

Los ejes se pueden hacer iguales con

```
plt.axis('equal')
```

Una abreviación del comando `meshgrid`, usando una notación indicial parecida a la que ocupa `r_`, es

```
X, Y = np.mgrid[-5:5:11j, -5:5:11j]
```

`mgrid` funciona también con más dimensiones; de hecho, `mgrid` se prefiere en lugar de `meshgrid`, el cual se provee más para compatibilidad con MATLAB.

Otra versión interesante es `ogrid`, que no rellena las matrices resultantes. Sin embargo, es igual de útil, debido al broadcasting:

```
X, Y = np.ogrid[-5:5:11j, -5:5:11j]
Z = X*Y
X
Y
```

8.3. Campos vectoriales

Los campos vectoriales se pueden dibujar con la función `quiver`¹. Esta función acepta 4 matrices, X y Y que dan las coordenadas X y Y de los puntos de la malla donde se especifica el campo, y U y V , que son las componentes de los vectores que dibujar en cada punto.

Por ejemplo, podemos dibujar un campo radial al poner

```
X, Y = np.mgrid[-5:5:11j, -5:5:11j]
plt.quiver(X, Y, X, Y)
```

ya que en el punto (x,y) de la malla ponemos el vector (x,y) . Lo podemos colorear según el valor de la distancia del origen con

¹Eso es una broma nerd: “quiver” quiere decir “carcaj” (un objeto donde se guardan las flechas).

```
X, Y = np.mgrid[-5:5:11j, -5:5:11j]
R = np.sqrt(X*X + Y*Y)
plt.quiver(X, Y, X, Y, R)
```

Algunos campos más interesantes son

```
plt.quiver(X, Y, Y, X)
plt.quiver(X, Y, Y, -X)
```

Resulta que se ven un poco “deformados”, ya que cada flecha se dibuja empezando en el punto dado de la malla. Se ve más ordenado al colocar el punto medio de la flecha en el punto de la malla:

```
quiver(X, Y, Y, -X, R, pivot='middle')
```

8.4. Gráficas en 3D

En versiones recientes de `matplotlib`, hay soporte limitado (aunque cada vez menos limitado) para gráficas en 3D. El módulo relevante es `mplot3d`.

Se pueden dibujar, entre otras cosas, líneas:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

t = np.arange(0, 10, 0.01)
x = np.cos(t)
y = np.sin(t)
z = t

ax.plot(x, y, z)
```

La gráfica se puede rotar en 3D, y se pueden agregar etiquetas, etc., de la manera usual.

También se pueden dibujar superficies:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

X = np.arange(-5, 5, 0.25)
Y = np.arange(-5, 5, 0.25)
X, Y = np.meshgrid(X, Y)
R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)
ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap=cm.jet)

plt.show()
```

8.5. Animaciones

Las gráficas hechas en `matplotlib` se pueden animar. En las versiones desde 1.1 de `matplotlib`, hay una mejor manera de hacer esto, que incluso permite guardar las animaciones como archivos tipo película. Desgraciadamente, ¡la manera anterior de animar dejó de funcionar!

```

import numpy as np
from matplotlib import pyplot as plt
from matplotlib import animation

fig = plt.figure() # crea una figura nueva
ax = plt.axes(xlim=(0, 2), ylim=(-2, 2))
line, = ax.plot([], [], lw=2)

# initialization function: plot the background of each frame
def init():
    line.set_data([], [])
    return line,

def animar(i):
    x = np.arange(-5., 5., 0.01)
    y = np.sin(x - i/200.)
    line.set_data(x, y)
    return line,

anim = animation.FuncAnimation(fig, animar, init_func=init,
                               frames=200, interval=20, blit=True)

# blit=True redibuja las partes que han cambiado

anim.save('anim_sen.mp4', fps=30)

plt.show()

```

8.6. Interacción con el mouse y el teclado

Se puede interactuar con el mouse y con el teclado de una manera relativamente sencilla.

Para interactuar con el mouse, creamos una función que se llamará cuando `pylab` note que hay un evento que involucre el mouse. Esta función toma un solo argumento, que suele llamarse `event`. `pylab` manda a la función un objeto que representa el evento, incluyendo cuál botón se presionó y en que posición de la gráfica. La función se registra ante `pylab` con la función `connect`.

Lo mismo tiene para el teclado. Así que el código más sencillo es el siguiente. Nótese que es necesario que haya una gráfica pre-existente antes de poder interactuar con ella.

```

from pylab import *

def teclado(event):
    print "Se tecleo %s en la posicion (%g, %g)" % event.key, event.xdata, event.ydata

def mouse(event):
    if event.button != 1:
        return
    x,y = event.xdata, event.ydata
    print "Se oprimio el boton izquierdo en (%g, %g)" % (x, y)

x = arange(10)
y = x*x
p, = plot(x, y)

```

```
show()

connect('button_press_event', mouse)
connect('key_press_event', teclado)
```

Ejercicio: Escribe un programa que recibe clics del ratón y coloca cargas eléctricas en las posiciones correspondientes. Cada vez que se agregue una carga nueva, el programa calcula el campo eléctrico y las líneas del campo nuevos correspondientes, y los dibuja.

Capítulo 9

Animaciones sencillas con Visual Python

En este capítulo, veremos un paquete, Visual Python, que permite hacer animaciones en 3 dimensiones, en tiempo real, de una manera sencillísima. ¡Casi vale la pena aprender Python solamente para eso! Evidentemente es excelente para enseñar conceptos básicos de la física; de hecho, fue diseñado principalmente para este fin.

9.1. El paquete Visual Python

La biblioteca de Visual Python (de aquí en adelante, “Visual”) se carga con

```
from visual import *
```

La manera más fácil de entender cómo utilizarlo es a través de un ejemplo.

Empecemos creando una esfera:

```
s = sphere()
```

Podemos ver `sphere()` como una función que llamamos para crear un objeto tipo esfera. (De hecho sí es una función, un *constructor* que construye un objeto de tipo `sphere`.) Para poder manipular este objeto después, se lo asignamos el nombre `s`.

Al crear objetos en Visual Python, *se despliegan por automático*, y ¡en 3 dimensiones! Como se puede imaginar, hay una cantidad feroz de trabajo abajo que permite que funcione eso, pero afortunadamente no tenemos que preocuparnos por eso, y simplemente podemos aprovechar su existencia.

¿Qué podemos hacer con la esfera `s`? Como siempre, `ipython` nos permite averiguarlo al poner `s.<TAB>`. Básicamente, podemos cambiar sus *propiedades internas*, tales como su color, su radio y su posición:

```
s.color = color.red # o s.color = 1, 0, 0 (cantidad de rojo, verde, azul)
s.radius = 0.5
s.pos = 1, 0, 0
```

Aquí, `s.pos` es un vector, también definido por Visual, como podemos ver al teclear `type(s.pos)`. Los vectores en Visual son diferentes de los que provee `numpy`; en particular, los de Visual siempre tienen 3 componentes, ya que se utilizan en Visual para describir un modelo del mundo tres-dimensional.

Por defecto, las gráficas en Visual se pueden rotar en 3 dimensiones con el ratón, al arrastar con el botón de derecho puesto, y se puede hacer un acercamiento (zoom) con el botón central.

Ahora podemos construir diferentes tipos de objetos incluidos en Visual, incluyendo `box` y `cylinder`.

9.2. Animaciones

Ahora llega lo bueno. ¿Cómo podemos hacer una animación? De hecho, ¿qué es una animación?

Una animación no es más que una secuencia de imágenes, desplegadas suficiente rápidamente una tras otra para que el ojo humano vea una secuencia ininterrumpida de movimiento (aproximadamente 25 cuadros por segundo). Así que eso es lo que tenemos que hacer: mover un objeto repetidamente y rápidamente.

```
s = sphere()
b = box()
for i in range(10000):
    rate(100)
    s.pos = i/1000., 0, 0
```

Aquí, la función `rate` es una función de Visual que limita en número de cuadros que se dibujan por segundo, por si la máquina es demasiado rápida. Nótese que `numpy` también incluye una función (con funcionalidad distinta) con este nombre, por lo cual si se carga `numpy`, conviene cargarlo antes de Visual, u ocupar la otra manera de importarlo (`import numpy as np`).ls /rate

9.3. Agregando propiedades a objetos

Supongamos que queremos pensar en nuestra esfera como una pelota. Entonces la pelota tendrá no solamente una posición, sino también una velocidad. Se lo podemos crear así:

```
pelota = sphere()
pelota.vel = vector(1,0,0)
```

Nótese que se tiene que poner explícitamente `vector`, ya que sino sería una *n*-ada (tupla). Ahora quedó definida la velocidad de la pelota como otra propiedad interna.

9.4. La escena

Por defecto, hay una escena llamada `scene`. Sus propiedades internas determinan, por ejemplo, el centro al cual apunta la cámara (`scene.center`) y si se autoescala la vista cuando los objetos en la escena se mueven (`scene.autoscale`).

Eso es básicamente todo lo que hay que saber de Visual Python. También es posible interactuar con el teclado, extraer las coordenadas del ratón, etc.

Capítulo 10

Cálculos de cómputo científico con scipy

Ya hemos visto que `numpy` provee ciertos submódulos que permiten llevar a cabo tareas avanzadas de cómputo científico, que consisten en interfaces bonitos a códigos pre-existentes.

El paquete `scipy`, que se basa en los `arrays` de `numpy`, provee una colección mucho más completa de rutinas para llevar a cabo una amplia gama de tareas básicas de cómputo científico.

Algunos de los submódulos más importantes de `scipy` incluyen

- `fftpack` – transformada rápida de Fourier
- `integrate` – integración de funciones y de ecuaciones diferenciales ordinarias
- `interpolate` – interpolación
- `linalg` – álgebra lineal
- `optimize` – optimización y raíces de funciones
- `signal` – procesamiento de señales
- `sparse` – rutinas para matrices escasas o dispersas (i.e. con muchas entradas 0); incluye rutinas para valores y vectores propios
- `special` – funciones especiales
- `stats` – funciones estadísticas

Al importar `scipy` con

```
import scipy as sp
```

se incluyen automáticamente todas las funciones de `numpy` en el espacio de nombres `sp`.

10.1. Álgebra lineal

10.2. Cálculos numéricos con `scipy`

El paquete `scipy` provee una colección de herramientas para llevar a cabo tareas numéricas, como son los siguientes submódulos:

Veamos algunos ejemplos. Para utilizar las funciones especiales, hacemos

```
from scipy import special
for i in range(5):
```

```
special.gamma(3)
special.gamma(3.5)
```

```
x = arange(-2, 2, 0.05)
```

```
for i in range(5):
    plot(x, map(special.hermite(i), x))
```

Para integrar la ecuación de Airy (un ejemplo de la documentación de `scipy`)

$$\frac{d^2 w}{dz^2} - zw(z) = 0, \quad (10.1)$$

podemos poner

```
from scipy.integrate import odeint
from scipy.special import gamma, airy
y1_0 = 1.0/3** (2.0/3.0) /gamma (2.0/3.0)
y0_0 = -1.0/3** (1.0/3.0) /gamma (1.0/3.0)
y0 = [y0_0, y1_0]
```

```
def func(y, t):
    return [t*y[1], y[0]]
```

```
def gradiente(y,t):
    return [[0,t], [1,0]]
```

```
x = arange(0,4.0, 0.01)
t = x
ychk = airy(x)[0]
y = odeint(func, y0, t)
y2 = odeint(func, y0, t, Dfun=gradiente)
```

```
print ychk[:36:6]
print y[:36:6,1]
print y2[:36:6,1]
```

En la segunda llamada a `odeint`, mandamos explícitamente la función que calcula la derivada (gradiente) de `f`; en la primera llamada, no fue necesario contar con una función que calculara eso.

Nótese que la función `odeint` maneja vectores como *listas*.

Encontrar raíces de funciones es más o menos sencillo:

```
def f(x):
    return x + 2*cos(x)

def g(x):
    out = [x[0]*cos(x[1]) - 4]
    out.append(x[1]*x[0] - x[1] - 5)
    return out

from scipy.optimize import fsolve
x0 = fsolve(func, 0.3)
x02 = fsolve(func2, [1, 1])
```


10.3. pygsl

El GNU Scientific Library (GSL) es una de las mejores bibliotecas de software libre para hacer cómputo científico hoy día. La biblioteca de Python `pygsl`¹ provee un “wrapper” (un interfaz delgado al estilo Python) para usar la biblioteca.

Por ejemplo, tiene funciones para métodos de Runge–Kutta de distintos tipos (que no hay en `scipy`).

¹Nótese que actualmente no hay ningún paquete disponible en los repositorios de Ubuntu para `pygsl`, por lo cual se tiene que bajar desde `pygsl.sourceforge.net`.

Capítulo 11

Visualizaciones en 3D con `mayavi2`

Una de las bibliotecas / programas que se ha desarrollado por Enthought es `mayavi2`. Es un entorno para la “visualización fácil e interactiva.”^{en} 3D, desarrollado por Enthought, y es software libre.

Hay tres maneras de usar `mayavi2`:

1. Como programa independiente
2. Scripting desde Python con `mlab`, estilo `matplotlib`
3. Con interfaz mas complicado

Ocupa un modelo tipo “pipeline”, que tiene tres niveles:

1. Cargar datos y guardar en fuente de datos
2. Procesar datos (opcionalmente) con filtros
3. Visualizar con módulos de visualización

Así se pueden visualizar los mismos datos de distintas maneras

Todos los objetos pertenecen a una escena, donde se hace la visualización en 3D.

11.1. Usando `mayavi2` de manera interactiva

Para usar `mayavi2` de manera interactiva desde `ipython`, es necesario usar la opción¹

```
ipython -wthread
```

Hay un entorno fácil de usar estilo `pylab`, que se llama `mlab`, lo cual se importa como sigue:

```
from enthought.mayavi import mlab
```

Hay distintos ejemplos que se pueden correr cuyos comandos empiezan con `test_`. Para tener una lista, en `ipython` se puede ejecutar

```
mlab.test_<TAB>
```

Aquí va un ejemplo tomado de la documentación:

¹Nótese que las opciones cambian por completo en la version 0.11 de `ipython`.

```

from numpy import pi, sin, cos, mgrid
dphi, dtheta = pi/250.0, pi/250.0
[phi,theta] = mgrid[0:pi+dphi*1.5:dphi,0:2*pi+dtheta*1.5:dtheta]
m0 = 4; m1 = 3; m2 = 2; m3 = 3; m4 = 6; m5 = 2; m6 = 6; m7 = 4;
r = sin(m0*phi)**m1 + cos(m2*phi)**m3 + sin(m4*theta)**m5 + cos(m6*theta)**m7
x = r*sin(phi)*cos(theta)
y = r*cos(phi)
z = r*sin(phi)*sin(theta)

# Verlo:
s = mlab.mesh(x, y, z)
mlab.show()

```

Nótese que `meshgrid` o `mgrid` son igual de importantes que en `pylab`.

Para manipular la figura generada, se usa el ratón como sigue: el botón izquierdo rota, el de en medio traslada, y el derecho hace un acercamiento.

Para limpiar la figura actual, se ocupa

```
mlab.clf()
```

11.2. Comandos básicos

Los comandos de `mayavi2` funcionan con tres arreglos de números o con mallas generadas con `meshgrid`.

La gráfica tal vez más sencilla es la siguiente:

```

from numpy import rand
x = rand(10)
y = rand(10)
z = rand(10)

# alternativo:
# x, y, z = rand(3, 100)

mlab.points3d(x,y,z)

```

Se pueden modificar los colores y tamaños de los puntos a través de un cuarto argumento, que es otro arreglo que da el valor:

```

c = rand(10)

mlab.points3d(x,y,z,c)
mlab.plot3d(x,y,z) # unir con líneas

mlab.plot3d? # muestra las opciones

mlab.plot3d(x,y,z, tube_radius = 0.01) # se dibujan con tubos
mlab.plot3d(x,y,z, c)

```

11.3. Animaciones

Hay una manera parecida que la de `pylab` para llevar a cabo animaciones –es necesario reemplazar los datos adentro de los objetos:

```

p = mlab.points3d(x,y,z,c) # crear un objeto
ms = p.mlab_source # su "fuente"

```

Se puede usar `ms.reset` en lugar de `ms.set` si el tamaño de los datos cambia.

Esto evita la necesidad de recalcular toda la grafica, y así acelera la animación.

Aquí he una lista de algunos de los comandos disponibles:

Para datos 1D: `points3d; plot3d`

Para datos 2D: `imshow; surf; contour_surf; mesh` (superficie dada por 3 arreglos `x;y;z` de las coordenadas); `barchart; triangular_mesh`

Para datos 3D: `contour3d; quiver3d` (`x; y; z` de posición; `y u; v; w` de velocidad); `flow` (trayectorias de partículas).

Nótese que `contour3d` y `flow` requieren datos ordenados.

11.4. Ejemplo: visualizar un campo escalar en 3D

```
import numpy as np
x, y, z = np.ogrid[-10:10:20j, -10:10:20j, -10:10:20j]
s = np.sin(x*y*z) / (x*y*z)
```

```
mlab.contour3d(s) # iso-superficies
```

El problema es que las superficies de afuera esconden las de adentro.

Entonces podemos usar un rendero de volumen: cada “voxel” tiene un color parcialmente transparente:

```
mlab.pipeline.volume(mlab.pipeline.scalar_field(s))
```

Podemos cambiar la opacidad:

```
mlab.pipeline.volume(mlab.pipeline.scalar_field(s), vmin=0, vmax=0.8)
```

Agregamos cortes transversales:

```
mlab.pipeline.image_plane_widget(mlab.pipeline.scalar_field(s),
plane_orientation='x_axes', slice_index = 10,)
mlab.outline()
```

Los planos se pueden mover con el raton

Más complicado aún:

```
src = mlab.pipeline.scalar_field(s)
mlab.pipeline.iso_surface(src, contours=[s.min()+0.1*s.ptp(), ], opacity=0.1)
mlab.pipeline.iso_surface(src, contours=[s.max()-0.1*s.ptp(), ],)
mlab.pipeline.image_plane_widget(src,
                                plane_orientation='z_axes',
                                slice_index=10,
                                )
```


Parte III

Programación orientada a objetos

Capítulo 12

Programación orientada a objetos: clases

En este capítulo, veremos uno de los temas claves del cómputo moderno: la *programación orientada a objetos*. Aunque pueda sonar complicado, todos ya somos expertos en la materia, ya que hemos estado utilizando los objetos de manera implícita en todo el curso, dado que *cualquier* cosa en Python (variables, listas, funciones, módulos, etc.) es un objeto.

En este capítulo, veremos cómo nosotros podemos definir nuestros propios tipos de objetos nuevos, y cómo sirve para hacer coincidir mejor nuestro modelo matemático de algún problema científico con el programa que lo representa en la computadora.

12.1. El problema: variables que están relacionadas conceptualmente, pero no en el código

Consideremos un sistema clásico de la física: un número N de partículas que se mueven en 2 dimensiones. En el modelo físico–matemático, podemos hablar de entidades, que son las partículas, las cuales tienen *propiedades internas*, incluyendo su (propia) posición, su velocidad y su masa. Si pensamos, por ejemplo, en un gas ideal, entonces las partículas no interactúan entre ellas, pero se pueden mover (avanzar su posición en el tiempo según su velocidad); para esto, ocuparemos para sencillez el método de Euler.

Para una partícula, podríamos definir sus variables simplemente como sigue:

```
x = y = 0.0    # posicion
vx = vy = 1.0  # velocidad
```

Un paso de Euler de tamaño dt se puede implementar con

```
dt = 0.1
x += dt * vx;
y += dt * vy;
```

Pero ahora, ¿qué hacemos si necesitamos otra partícula más? Podríamos poner

```
x1 = y1 = 0.0
x2 = y2 = 0.0
vx1 = vy1 = 1.0
vx1 = vy1 = 1.0
```

Si las partículas también tienen masas y colores, entonces se vuelve muy tedioso, ya que tenemos que actualizar todo dos veces para cada propiedad nueva:

```
m1 = m2 = 0.0;
c1 = c2 = 0.0;
```

Para muchas partículas podríamos emplear arreglos (listas, en Python), que reduciría el trabajo.

Pero ahora podríamos imaginarnos que por alguna razón queremos agregar otra partícula especial (por ejemplo, un coloide en un fluido), o incluso duplicar toda la simulación (dos gases en cajas independientes). Entonces tendríamos que duplicar a mano todas las variables, cambiando a la vez sus nombres, lo cual seguramente conducirá a introducir errores, y además es muy tedioso.

Sin embargo, *conceptualmente* tenemos muchas variables que están relacionadas: todas *pertenecen* a una partícula. Hasta ahora, no hay manera de expresar esto en el programa.

12.2. La solución: encapsulación de datos en objetos (“clases”)

Lo que queremos hacer, entonces, es reunir todo lo que corresponde a una partícula dada en un nuevo *tipo de objeto*, llamado `Particula`. Luego podremos decir que `p1` y `p2` son `Particulas`, o incluso hacer un arreglo (lista) de `Particulas`.

Toda la información que le corresponde a una partícula dada estará contenida adentro del objeto correspondiente; esta información formará parte del objeto no sólo conceptualmente, sino también en la representación en el programa.

Podemos pensar en un objeto, entonces, como un tipo de “caja negra”, que tiene propiedades internas, y que puede interactuar de alguna manera con el mundo externo. No es necesario saber o entender qué es lo que hay adentro del objeto para entender cómo funciona. Se puede pensar que corresponde a una caja con palancas, botones y luces: las palancas y los botones proveen una manera de darle información o instrucciones a la caja, y las luces dan información de regreso al mundo externo.

Para implementar eso en Python, se declara una *clase*¹ llamada `Particula`. Esto declara la estructura que posee el nuevo tipo de objeto (pero no crea ningún objeto de este tipo):

```
class Particula(object):
    x = 0.0
    v = 0.0
```

La palabra `object` entre paréntesis indica que estamos “heredando” de un tipo de objeto llamado `object`. Esto corresponde a las clases al “estilo nuevo”; si se omite `object`, se tiene una clase al “estilo viejo”, donde no funcionan ciertas cosas.

Luego, se crea una *instancia* de esta clase, llamada `p`—es decir, el nombre `p` se refiere a un nuevo objeto

```
p = Particula()
p.<TAB> # muestra una lista del "contenido" de p
p.x
p.v
```

Nótese que `p` tiene *adentro* dos propiedades, o atributos, llamados `x` (su posición) y `v` (su velocidad). Podemos interpretar `p.x` como una `x` que le *pertenece* a `p`.

Si ahora hacemos

```
p2 = Particula()
p2.x
p2.v
```

entonces tenemos una variable completamente *distinta* que también se llama `x`, pero que ahora le pertenece a `p2`, que es otro objeto de tipo `Particula`. De hecho, más bien podemos pensar que `p2.x` es una manera de escribir `p2_x`, que es como lo hubiéramos podido escribir antes, que tiene la bondad de que ahora también tiene sentido pensar en el conjunto `p` como un todo.

¹Usado en el sentido de “tipo de” cosas: hay una clase de cosas llamada `Particulas`.

Notemos que ya estamos acostumbrados a pensar en cajas de este tipo en matemáticas, al tratar con vectores, matrices, funciones, etc. Ahora tenemos una manera de *encapsular los datos* dentro de un objeto de esta forma también en Python².

12.3. Métodos: acciones que llevan a cabo los objetos

Hasta ahora, una clase actúa simplemente como una caja que contiene datos. Pero objetos no sólo tienen información, sino también pueden *llevar a cabo acciones*. Para hacerlo, también se pueden definir *funciones* que le pertenecen al objeto –en este contexto, se llaman *métodos* del objeto. Simplemente se definen las funciones adentro de la declaración de la clase:

```
class Particula(object):
    x = 0.0
    v = 1.0

    def mover(self, dt):
        self.x += self.v*dt

p = Particula()
type(p)
print p.x
p.mover(0.1)
print p.x
```

Nótese que los métodos de las clases siempre llevan un argumento extra, llamado `self`, que quiere decir “sí mismo”. Las variables que pertenecen a la clase, y que se utilizan adentro de estas funciones, llevan `self.`, para indicar que son variables que forman parte de la clase, y no variables globales. Podemos pensar en las variables internas a la clase como variables “pseudo-globales”, ya que están accesibles desde cualquier lugar *adentro* de la clase.

Se pueden agregar nuevas variables internas (*atributos*) a un objeto desde cualquier lado del programa:

```
p.masa = 3.
```

•

12.4. Función inicializadora (“constructor”)

Cuando creamos una instancia de un objeto, muchas veces queremos *inicializar* el objeto al mismo tiempo, es decir, pasarle información sobre su estado inicial. En Python, esto se hace a través de una función inicializadora (también llamado “constructor”), como sigue:

```
class Particula(object):
    def __init__(self, xx=0.0, vv=1.0):
        self.x = xx
        self.v = vv

    def mover(self, dt):
        self.x += self.v*dt

p1 = Particula()
print p1.x
p2 = Particula(2.5, 3.7)
print p2.x
```

²Algo equivalente en C para encapsular datos es `struct`.

```
p1.mover(0.1)
p2.mover(0.1)
print p1.x, p2.x
```

Nótese que la función inicializadora debe llamarse `__init__`, con dos guiones bajos de cada lado del nombre `init`. Esta función se comporta como una función cuyo nombre es exactamente el nombre de la clase, y se llama como tal. Además, puede tomar argumentos que se utilizan para inicializar las variables de la clase, que son los argumentos que se pasan a `__init__`.

La función inicializadora, de existir, se llama automáticamente para inicializar cada instancia de la clase, al momento de crearla o construir la instancia (de ahí el nombre “constructor”). Esta función regresa la variable `self` (implícitamente) que se utiliza después. Nótese que en este ejemplo, el único lugar donde se crean variables adentro de la clase es en la función `__init__`, ya que cuando se crea una variable adentro de la clase, esta variable está disponible en cada parte de la clase.

12.5. Métodos internos (escondidos) de las clases

Las clases pueden ocupar varios métodos internos para proveer un interfaz más bonito para el usuario. Estos métodos están escondidos y no están diseñados para utilizarse directamente por el usuario. Por ejemplo, al poner `print p1` en el último ejemplo, sale algo así como

```
<__main__.Particula instance at 0x2849cb0>
```

Podemos hacer que salga algo más útil al proveer adentro de la clase una función `__str__`:

```
class Particula(object):
    def __init__(self, xx=0.0, vv=1.0):
        self.x = xx
        self.v = vv

    def mover(self, dt):
        self.x += self.v*dt

    def __str__(self):
        return "Particle con propiedades (%g, %g)" % (self.x, self.v)

p1 = Particula()
print p1
```

Si no ponemos `print p1`, sino sólo `p1`, entonces nos da la representación interna del objeto. Aquí también podemos regresar algo más útil, al definir la función interna `__repr__` de la clase:

```
def __repr__(self):
    msg = "Particle(%g, %g)" % (self.x, self.v)
```

La función `__repr__` (“representación”) debe regresar una representación que se puede utilizar para construir un objeto del mismo tipo.

También podemos definir funciones que permiten que podamos llevar a cabo operaciones aritméticas etc. con objetos de este tipo, es decir, podemos *sobrecargar* los operadores para que funcionen con nuestro nuevo tipo. Eso es lo que pasa con los `array` de `numpy`, por ejemplo. Sin embargo, en el caso de una partícula, no parece tener mucho sentido definir, por ejemplo, una función `+`.

Ejercicio: Crea un objeto para representar a una partícula en 2D. ¿Cómo se puede crear una “nube” de partículas, es decir una colección de un gran número de ellas? Crea una clase que representa una tal nube.

12.6. Partículas en n dimensiones

¿Cómo podríamos crear una partícula cuya posición y velocidad fueran vectores n dimensionales? Al revisar el código, en ninguna parte se utiliza el hecho de que x y v tienen que ser números –pueden ser de cualquier tipo.

Sin embargo, en la función `mover`, se utiliza el código

```
self.x += self.v*dt
```

Por lo tanto, es crucial que ocupemos objetos para los cuales están definidos las operaciones `+=` y `*`. Las listas de Python por lo tanto no servirán. Sin embargo, ¡los arrays de numpy sí!

Así que *sin modificar el código ya escrito en lo más mínimo*, podemos ¡usar el mismo código con arrays al inicializar los objetos con arrays!

Por ejemplo:

```
from numpy import *

x = r_[1., 2, 3]
v = r_[.5, .5, 0]

p = Particula(x, v)
p.mover(1)
print p.x
```

12.7. Herencia

Las clases también pueden formar jerarquías al utilizar la *herencia*, que quiere decir que una clase es un “tipo de”, o “subtipo de” otro. Así, podemos reutilizar código.

En algunos lenguajes (por ejemplo, C++), la única manera de hacer que objetos diferentes tengan un comportamiento común es a través de herencia. En Python, se puede simplemente crear una colección de objetos, pero la herencia unifica el interfaz.

Por ejemplo, supongamos que tenemos un programa para dibujar distintos tipos de objeto. Los objetos se guardan en una lista y quisiéramos poder iterar sobre la lista para dibujar todos los objetos, agregar objetos a la lista, etc.

Por el momento, tendríamos que hacer algo así (en pseudo-código):

```
def dibujar(lista_objetos):
    for objeto in objetos:
        tipo = encontrar_tipo(objeto)
        if tipo==0:
            dibujar_rectangulo()
        else if tipo==1:
            dibujar_circulo()

def mover(objeto):
    tipo = encontrar_tipo(objeto):
    if tipo==0:
        mover_rectangulo(objeto)
    else if tipo==1:
        mover_circulo(objeto)
```

Lo que realmente necesitamos es una manera de decir que `Rectangulo` y `Circulo` son *subtipos de* algo más general, que podemos llamar `Forma`. Lo hacemos a través de *herencia*: `Rectangulo` y `Circulo` *heredan* todas las propiedades de `Forma`

```

class Forma(object):
    def __init__(self, x=0.0, y=0.0):
        self.x = x
        self.y = y

    def dibujar(self):
        print "En (%s, %s)" % (self.x, self.y)

class Rect(Forma):
    def __init__(self, x=0.0, y=0.0):
        Forma.__init__(self, x, y)
        self.width = 1.
        self.height = 2.

    def dibujar(self):
        print "Rectangulo",
        Forma.dibujar(self)
        print "Width =", self.width, "; height = ", self.height

class Circulo(Forma):
    def __init__(self, x=0.0, y=0.0):
        Forma.__init__(self, x, y)
        self.radius = 3.

    def dibujar(self):
        print "Circulo",
        Forma.dibujar(self)
        print "Radius = ", self.radius

```

Ejercicio: Haz una colección de objetos al azar y “dibújalos”.

Ejercicio: Utiliza `patches` de `matplotlib` para dibujar los objetos de a de veras.

Nótese que `matplotlib` tiene una función `pyplot.ginput(n)` que deja al usuario escoger n puntos de manera interactiva.

Sin embargo, como en Python se pueden crear listas de objetos de distintos tipos, y “todo funciona”, la herencia no es tan necesaria como en otros lenguajes tipo C++.

Parte IV

Otros paquetes y herramientas

Capítulo 13

Cálculo simbólico con Python

Hasta ahora en el curso, hemos estado trabajando con cálculos numéricos. Sin embargo, en muchos problemas de cómputo científico, es útil poder llevar a cabo manipulaciones simbólicas pesadas con la computadora. En el mundo del software comercial, Mathematica es una buena opción para esto. En cuanto a software libre, y más en particular disponible desde Python, hay dos opciones principales: (i) el paquete `sympy`, que es un paquete normal de Python que permite crear y manipular objetos simbólicos; y (ii) el entorno completo SAGE, que pretende ser un competidor de Mathematica, proviendo un entorno completo que integra muchos paquetes de software libre matemáticos, y utiliza Python como el “pegamento” para que todos funcionen juntos.

13.1. El paquete `sympy`

El paquete `sympy` (“symbolic Python”) provee una colección de rutinas para hacer cálculos simbólicos.

Para indicar que un nombre se refiere a una variable simbólica, se tiene que indicar explícitamente:

```
from sympy import *
x, y, z = symbols('xyz')
k, m, n = symbols('kmn', integer=True)
f = Function("f")
```

Luego estos objetos se pueden manipular en expresiones simbólicas:

```
x + x
z = (x + y) ** 2
z.expand()          # expandir

z.subs(x, 1)        # sustituir

limit(sin(x) / x, x, 0) # calcular limites
limit(1 - 1/x, x, oo)

diff(sin(2*x), x)      # derivar
diff(sin(2*x), x, 3)

cos(x).series(x, 0, 10) # series de Taylor

integrate(log(x), x)    # integral indefinida
integrate(sin(x), (x, 0, pi/2)) # integral definida

f(x).diff(x, x) + f(x)
dsolve(f(x).diff(x, x) + f(x), f(x)) # resolver ecuacion diferencial
```

```
solve(x**4 - 1, x)
```

```
pi.evalf(50)      # evaluar numero flotante con precision arbitraria
```

Nótese que hay una versión de IPython diseñado explícitamente para hacer que la salida de `sympy` se vea más bonita, llamada `isympy`.

13.2. Entorno completo: SAGE

SAGE es un entorno completo para hacer matemáticas, disponible libremente desde la página <http://www.sagemath.org>. pretende reemplazar a programas del estilo de Mathematica, al proveer un entorno completo usando paquetes de software libre.

SAGE provee un interfaz usando Python a muchos paquetes libres para hacer matemáticas. Además, provee un interfaz disponible por el internet, a través de un navegador web, para interactuar con los “notebooks”.

Se corre con

```
sage      # linea de comandos
sage -notebook      # version web
```

Uno trabaja en Python, con algunas modificaciones cómodas, por ejemplo que $\wedge$ se redefine para indicar exponenciación, y $1/2$ se mantiene como un quebrado. Provee maneras de manipular objetos simbólicos tal como en `sympy` (de hecho, `sympy`, siendo un paquete de Python, puede utilizarse desde SAGE, tal como `numpy`, `scipy` y `matplotlib`. Aparte, tiene interfaces a muchos paquetes de software libre, aunque varios de ellos son muy especializados para distintos tipos de matemáticas puras.

En la parte simbólica, SAGE es parecido a `sympy`, pero utiliza `Maxima` internamente para la mayoría de las cosas. En la física, es común querer hacer cadenas de sustituciones. Hasta cierto punto, esto se puede lograr poniendo las sustituciones en un diccionario:

```
var('alpha beta x y z')
sustituciones = {alpha: x+y, beta: y + z}      # diccionario de
s = alpha + beta
ss = s.subs(sustituciones)
sustituciones[z] = x*y
sustituciones
ss = ss.subs(sustituciones)      # hay que sustituir multiples veces para terminar de sustituir too
```

[Nótese que se utiliza SHIFT+ENTER para que registre el comando en cada celda.]

Además, tiene una manera fácil de hacer cosas de manera interactiva:

```
var('x')
x0 = 0
f = sin(x)*e^(-x)
p = plot(f,-1,5, thickness=2)
dot = point((x0,f(x=x0)),pointsize=80,rgbcolor=(1,0,0))
@interact
def _(order=(1..12)):
    ft = f.taylor(x,x0,order)
    pt = plot(ft,-1, 5, color='green', thickness=2)
    html('$f(x)\;=\; %s$'%latex(f))
    html('$\hat{f}(x; %s)\;=\; %s\mathcal{O}(x^{\%s})$'%(x0,latex(ft),order+1))
    show(dot + p + pt, ymin = -.5, ymax = 1)
```

Esto sólo funciona en SAGE, aunque es Python por completo.

La sintaxis `@interact` es un *decorador* (“decorador”), que toma una función y la convierte en otra. [Se cubrirán esto en detalle en el curso avanzado.]

Nótese que también se pueden usar clases (y cualquier otro pedazo de Python).

Capítulo 14

Depurador y perfilador

14.1. Depurador

Python tiene un depurador (“debugger”) integrado, `pdb`. Se puede usar de manera interactiva en IPython con

```
ipdb on
...      codigo
ipdb off
```

14.2. Perfilador

Se puede hacer un perfilamiento del código para optimizarlo con el paquete estándar `cProfile`, [Esto se discutirá con más detalle en el curso avanzado.]

Capítulo 15

Otros paquetes

15.1. Notebook de IPython

IPython tiene un notebook de HTML muy poderoso. Se corre con

```
ipython notebook
```

Se puede incluir tanto texto, como código, y los notebooks se pueden publicar en la red.

15.2. pandas

pandas es un paquete para manipular conjuntos de datos grandes de una manera más atractiva: se puede considerar como una envoltura (wrapper) de numpy, que permite asociar *nombres* con los distintos renglones y columnas de datos [tal como “dataFrame” en R].

Hay dos tipos básicos, para datos 1D (“Series”) y datos 2D (“DataFrame”).

```
import pandas as pd
from numpy.random import randn
from numpy import arrange

s = pd.Series(randn(5), index=list('abcde'))    # conjunto de datos 1D con nombres
s
```

Se pueden manipular los arreglos

```
s + s
s[1:] + s[:-1]
```

En el segundo ejemplo, vemos que hay pandas maneja los datos faltantes a través del valor especial NaN (“Not a Number”).

Datos 2D pueden tener nombres tanto para los renglones, como para las columnas:

```
d = {'one' : [1., 2, 3, 4],      # diccionario
      'two' : [5., 6., 7., 8] }

datos = DataFrame(d)

datos = DataFrame(d, index=list("abcde"))

datos['one']          # columna
```

```
datos.xs("a")      # renglon
datos.ix[3]        # seleccionar el renglon numero tal

datos['three'] = arange(3, 7)  # agregar column
```

Hay `Panel` y `PanelND` para arreglos 3D y n -dimensional, respectivamente, que funcionan de una manera parecida.

15.3. Operaciones con objetos en pandas

Se pueden hacer operaciones sobre los datos en un objeto de `pandas` de manera fácil, por ejemplo promedios en las distintas direcciones (es decir, sobre distintos ejes):

```
datos.mean(0)
datos.mean(1)
```